

Article

P3: Persistence-Aware Admission Control for DoS-Resilient BLE Resolvable Private Address Resolution

Shen Chong ¹, Longcun Wang ¹, Thi-Kien Dao ^{1,2} and Trong-The Nguyen ^{1,2,*}¹ School of Electronic Engineering, Fuzhou Institute of Technology, Fuzhou 350506, China; shenchongcn@gmail.com (S.C.); wanglc@gm.uit.edu.vn (L.W.); kiendt@uit.edu.vn (T.-K.D.)² Multimedia Communications Laboratory, University of Information Technology, VNU-Ho Chi Minh City, Ho Chi Minh City 70000, Vietnam

* Correspondence: thent@uit.edu.vn

Abstract

Bluetooth Low Energy (BLE) resolvable private addresses (RPAs) reduce passive tracking, but RPA-resolving creates receiver-side work: a scanner whose bonded identity set exceeds controller resolving-list capacity which can be forced into repeated host-side Identity Resolving Key (IRK) searches by floods of syntactically valid unknown private-address candidates. We formulate this as capacity-constrained resolving scheduling and propose P3, our persistence-aware admission-control scheduler for BLE RPA resolution, evaluated in its main form P3-Persist. After cache and resolving-list fast paths miss and the unresolved-work budget is exhausted, P3 grants a small reserve only to over-the-air RPA values that reappear. This uses visible repetition rather than an identity-correlated pre-resolution label. In paired simulations, identical visible traces produced identical initial pre-resolution decisions, but later modeled delay/defer features were distinguishable in some tested conditions; accordingly, no end-to-end side-channel-free claim is made. In the evaluated 20-seed, 1800 s simulation matrix ($N = 512$, $RL = 8$), P3-Persist keeps modeled unique-flood work in the same bounded approximately 45k AES-equivalent/min regime as budget-only limiting while increasing modeled legitimate resolution to 0.9569 under medium flood and 0.9557 under heavy flood, compared to 0.57–0.62 for budget-only baselines. A 20-seed epoch-loss-density experiment shows that service recovery is conditional rather than universal, and a denser adaptive-address search identifies an overlapping near-cap region around $m = 320$ and $r = 1$, with a highest confirmed mean of 146,196.29 AES-equivalent/min (96.69% of the modeled cap). These results establish bounded-work and conditional service-recovery behavior within the simulator; they do not establish production-firmware effectiveness, end-to-end side-channel freedom, on-device timing, energy, current, latency, or queueing behavior.



Academic Editor: Priyadarsi Nanda

Received: 17 June 2026

Revised: 16 July 2026

Accepted: 23 July 2026

Published: 28 August 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

Keywords: Bluetooth Low Energy; resolvable private address; denial-of-service resilience; admission control; Identity Resolving Key; RPA flooding; resource scheduling; persistence-aware scheduling; simulation-based evaluation

1. Introduction

Bluetooth Low Energy is now a default connectivity substrate for phones, watches, tags, health devices, industrial beacons, access-control peripherals, and sensor gateways [1–5]. Many of these deployments rely on BLE privacy features because an advertising address that remains stable over time is an obvious tracking identifier [6,7]. A resolvable private

address is intended to provide a practical compromise defined by the Bluetooth Core Specification: the address changes periodically, but trusted peers that hold the correct Identity Resolving Key (IRK) can still resolve it [8]. A passive observer that does not hold the IRK should not be able to link rotations purely from the address field [9]. This privacy mechanism also moves work into the receiver. A scanner must determine whether a private address corresponds to a known identity, and that determination is cheap only when the identity is already present in a controller resolving list or a recent cache [10,11]. Embedded controllers expose resolving-list capacity as a scarce resource [12]. Host stacks may store more bonded identities than the controller can resolve directly, especially in gateways, test benches, access-control hubs, or long-lived phones that have accumulated many pairings [13]. When an address misses the controller fast path, the host may need to search stored IRKs. The abstract cost of that search is an AES-equivalent resolving attempt, repeated until a matching IRK is found or the candidate set is exhausted [8,11].

Under ordinary traffic, this cost is usually acceptable. The problem becomes acute when the scanner receives many unknown RPA-like advertisements. An attacker does not need to recover an IRK, defeat BLE encryption, or impersonate a bonded device [14]. It is sufficient to inject many syntactically plausible private-address candidates that will not match local keys [15]. The receiver may then spend repeated host-side resolving effort on traffic that has no legitimate value. This is a denial-of-service vector at the private-address resolution layer: the protocol's privacy-preserving mechanism can be used to induce resource amplification [16]. The challenge is subtle because obvious shortcuts can be wrong. Prior BLE privacy studies have shown across address behavior, advertising payloads, allowlist behavior, and implementation responses that small differences can become side channels [7,13,17]. A defense that tries to classify an unknown RPA before normal resolving work may accidentally create a membership oracle: an attacker could infer whether a rotated address belongs to a bonded device by observing different handling [13,18]. Therefore, this paper does not use a Bloom filter or any other unknown-RPA prefilter to decide whether an unseen address is worth resolving [19]. The scientific boundary is that P3 schedules and bounds work; it does not identify unknown private addresses.

P3 addresses receiver-side resolving-work scheduling rather than BLE cryptography, tracking prevention, or general intrusion detection. It jointly manages a capacity-limited controller resolving list and bounded host fallback while reporting legitimate deferral. Figure 1 shows the shared cache, resolving-list, and host-fallback path for bonded, benign unknown, and attack advertisements.

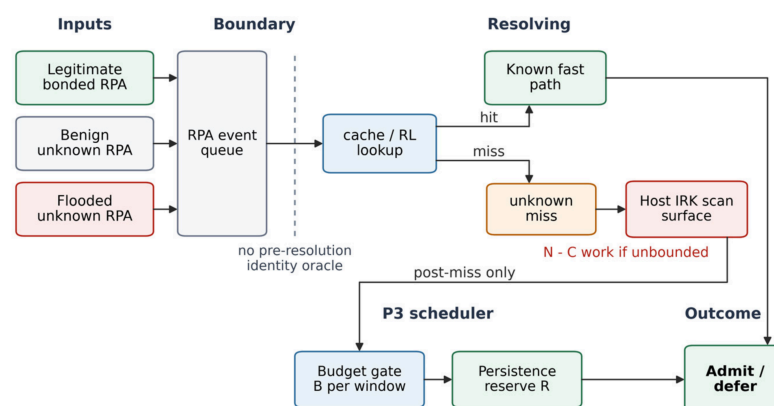


Figure 1. BLE private-address resolution as a capacity and DoS scheduling problem: bonded, benign unknown, and attack advertisements share cache, resolving-list, and host-fallback work paths.

Public Zephyr/NCS and BlueZ documentation exposes the host/controller, key store, resolving-list, and cache boundaries used to map the model to plausible stack

locations [10,11,20,21]. The implementation-context audit used Nordic nRF Connect SDK v3.2.1 with the bundled Zephyr 4.2.99 source tree. The BlueZ reference corresponds to BlueZ 5.84 documentation and was used for documentation context only; no BlueZ runtime was executed in this study. The current evidence remains simulation-based and stack-informed rather than a vendor-specific P3 firmware implementation. P3 targets a documented capacity-mismatch regime rather than all BLE deployments. Open-stack documentation and source paths show that controller resolving-list capacity can be smaller than the host bond/key store, so an ' $N > C$ ' state is technically possible in retained-bond centrals and gateways [10–12,21]. A building access-control hub with $N = 64$ retained badges and $C = 8$ controller entries illustrate the resulting 56-identity host-fallback set; it is not a prevalence estimate. The study therefore makes conditional claims for public-space scanners, shared-infrastructure centrals, access-control hubs, and similar systems that both satisfy ' $N > C$ ' and expose an open advertising channel. Beacon-only gateways, closed test centrals, and devices whose bonded identities fit in the controller list are outside scope. The headline $RL = 8$ setting is a constrained-controller anchor, while $RL = 16$ and $RL = 32$ are sensitivity points. No claim is made about how frequently this regime occurs across commercial deployments. This paper makes four contributions:

- It formulates BLE RPA resolution as a joint resolving-list capacity and RPA-flooding scheduling problem, separating this work from generic BLE tracking, energy optimization, and denial-of-service defense.
- It proposes P3-Persist, a persistence-aware reserve admission policy that uses only visible address repetition after the general unresolved-work budget is exhausted; capacity-aware IRK admission, positive caching, and budgeted admission remain supporting components rather than the novelty in isolation.
- It evaluates P3 against FullScan-Host, StaticRL, ZephyrCache, LRU/Frequency/RSSI-hint/Budget baselines, generic AdaptiveRateLimit, P3-NoPersist, and other module-disabled variants, reporting AES-equivalent attempts, attack amplification, legitimate resolution, false-deferral cost, reserve and threshold sensitivity, a paired modeled-observable privacy audit, a joint epoch-loss-benign-density experiment, and a dense local adaptive-attacker search.
- It provides a reproducible manuscript package that connects figures, tables, source CSVs, scripts, and the simulation configuration needed to reproduce the reported results.

The paper does not claim that caching, resolving-list replacement, or token-bucket-style limiting are new in isolation [11,22]. Its claim has two parts: under the privacy constraint that forbids identity-correlated pre-resolution classification [7,13,18], bonded-membership signals are removed from the pre-resolution scheduler (formalized as Proposition 0 in Section 4.8); and visible address repetition is the deployable discriminator used here along the axis where legitimate RPA epochs and amplification-oriented flooding differ. P3's persistence reserve acts on this signal while keeping admitted host work under an explicit budget-plus-reserve bound. The paired simulation supports identity-label invariance only for initial pre-resolution decisions, the joint-factor experiment limits service recovery to tested conditions, and the adaptive refinement identifies a tied near-cap local region rather than a global attacker optimum; Design Rationale 5 therefore remains an empirical service-work trade-off, not an end-to-end privacy or optimality result.

The rest of the paper is organized as follows. Section 2 reviews BLE privacy, RPA tracking, BLE security, energy modeling, and open-stack context. Section 3 defines the threat model and theoretical resolving-work model. Section 4 presents P3. Section 5 describes the evaluation design. Section 6 reports results. Section 7 discusses limitations and publication boundaries. Section 8 concludes.

2. Related Work, Background, and Motivation

2.1. BLE Privacy, RPA Semantics, and Tracking

BLE address rotation reduces direct tracking but does not remove payload-, behavior-, allowlist-, or implementation-based linkability [7,16–18,23–25]. In particular, allowlist-conditioned behavior can expose bonded membership [13,18]. These studies define P3’s privacy boundary: before normal IRK resolution, the scheduler may use receiver-local load state and visible address repetition, but it must not classify or externally signal whether an unknown RPA belongs to the bonded set.

2.2. BLE Security and Denial-of-Service

BLE security work covers large-scale scanning, spoofing, state-machine attacks, re-connection flaws, mesh security, and implementation-level denial of service [6,26–31]. P3 addresses a narrower layer: after cache and resolving-list misses, unknown private-address candidates can amplify host IRK-search work. Radio jamming, connection flooding, pairing abuse, GATT attacks, and mesh attacks remain outside scope because they do not answer how a receiver should bound post-miss resolving work without creating a membership oracle.

2.3. Energy and Embedded BLE Context

BLE energy modeling has shown that protocol choices, scan intervals, connection parameters, and cryptographic operations matter for constrained devices. Kindt et al. provide a useful example of modeling energy at the BLE protocol level [32]. Comparative BLE measurement studies also show that scanning, advertising, and protocol parameters can dominate low-power behavior in embedded settings [1,32,33]. These works motivate why receiver-side resolving work is relevant in embedded BLE systems. Section 6.5 defines the AES-equivalent cost abstraction and its measurement limits.

2.4. Open BLE Stacks and Practical Cache Behavior

Zephyr/NCS and BlueZ provide public implementation anchors for host/controller separation, IRK storage, resolving-list management, and positive caching [10,11,20,21]. ZephyrCache, P3-NoCache, and the budget ablations separate existing cache behavior from P3’s bounded post-miss admission contribution. Table 1 summarizes the resulting comparison.

Table 1. Comparison of representative BLE privacy/security and scheduling approaches against P3.

Approach (Representative Work)	Problem/Threat Focus	Manages RL Capacity	Bounded Host Work	Privacy-Safe Scheduler Use	Δ vs. P3
RPA/address tracking analysis (Becker et al. [24]; Celosia and Cunche [7])	Passive tracking via payload and address carry-over	No	No	N/A	Motivates why RPA matters; does not study resolving-load DoS or RL scheduling
Specification/allowlist traceability (Wu et al. [18]; Zhang and Lin [13])	Linkability from specification/implementation and allowlist behavior	No	No	N/A	Sets the no-membership-oracle constraint P3 must respect; proposes no scheduler
User-side privacy protection (Fawaz et al. [25])	Protecting BLE broadcast privacy at the user side	No	No	Yes	Protects broadcast privacy; not receiver-side resolving-list/AES work

Table 1. Cont.

Approach (Representative Work)	Problem/Threat Focus	Manages RL Capacity	Bounded Host Work	Privacy-Safe Scheduler Use	Δ vs. P3
Spoofing and state-aware defense (BlueShield [29]; BlueSWAT [31])	Spoofing detection/lightweight state security	No	No	Yes	Different attack surface; not resolving-layer work amplification
Implementation/link DoS (SweynTooth [27]; BLESa [28])	Crash, reconnection and packet-crafting DoS	No	No	N/A	Different DoS layer; P3 targets RPA-resolving work
Large-scale BLE scanning (BLEB [26])	Botnet-scale tracking and scanning	No	No	N/A	Supplies high-density flooding motivation; not a receiver-side defense
BLE security surveys (Lacava et al. [6]; Ghorl et al. [30])	Taxonomy of BLE threats and procedures	No	No	N/A	Background context; no RPA-resolving-list scheduling algorithm
Open-stack cache and RL (Zephyr [11]; Nordic NCS [21])	prpa_cache, host key lookup, capacity-bounded RL	fixed config only	No	Yes	Cache/RL exists; used as the Zephyr-Cache/StaticRL baselines, not claimed as novelty
Generic rate limiting, token bucket, and early dropping (RFC 2697 [22]; RED [34])	Bounding or shedding work at an interface	No	generic	N/A	Budget idea is generic and content-blind; P3 binds it to the unknown-RPA-resolving queue and adds persistence-aware service recovery
Probabilistic prefilter (Bloom filter [19])	Compact set-membership testing	No	No	No (unsafe here)	Cannot classify an unknown RPA without IRK/AES work; P3 deliberately avoids any prefilter
P3 (this work)	RPA-flooding resolving DoS under RL-capacity limits	Yes (activity-based admission)	Yes (unresolved-RPA + persistence reserve)	Initial-decision constraint only	Uses address repetition under Proposition 0's initial-decision identity-label-invariance constraint: same unique-flood bound as rate limiting (Proposition 1), arbitrary-attack bound under $B + R$ (Proposition 2), much higher legitimate service (Proposition 3), explicitly reported adaptive reserve-pressure boundaries (Proposition 4), and an empirical service-work trade-off within the evaluated admissible bounded-work class (Design Rationale 5)

N/A = not applicable.

Table 1 separates privacy constraints, BLE attack studies, existing cache/RL mechanisms, and generic work limiters. P3 combines the no-membership-oracle constraint with capacity-aware fast-path placement, bounded host work, and repetition-based reserve admission. Figures 2 and 3 visualize this admissible signal boundary and related-work position.

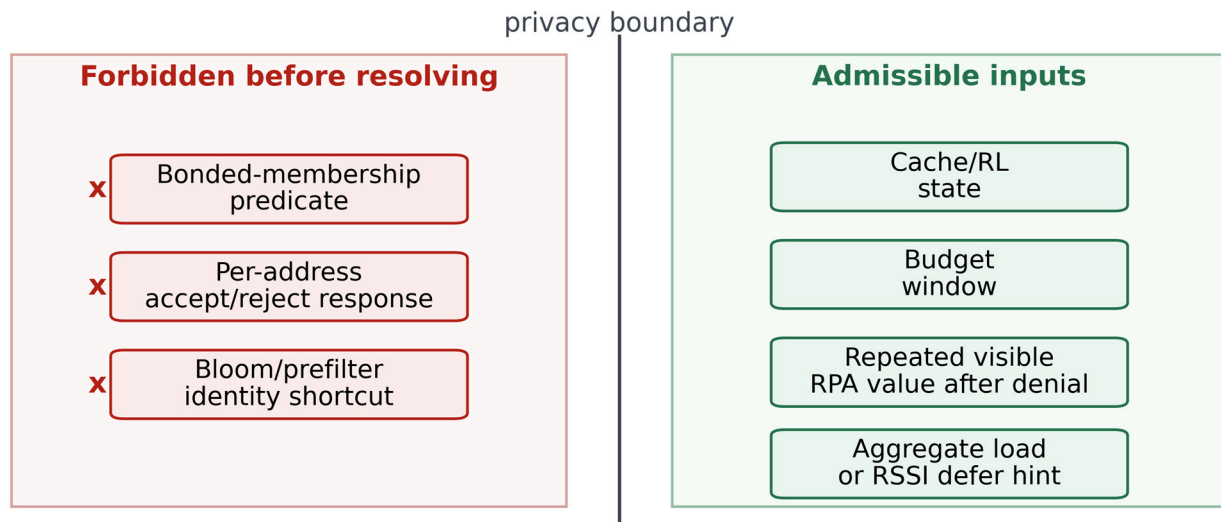


Figure 2. BLE privacy boundary and admissible scheduling signals: P3 excludes bonded-membership predicates and uses only receiver-local state and visible stream repetition before resolving. The red X marks indicate signals or actions that are forbidden before resolution under the privacy boundary.

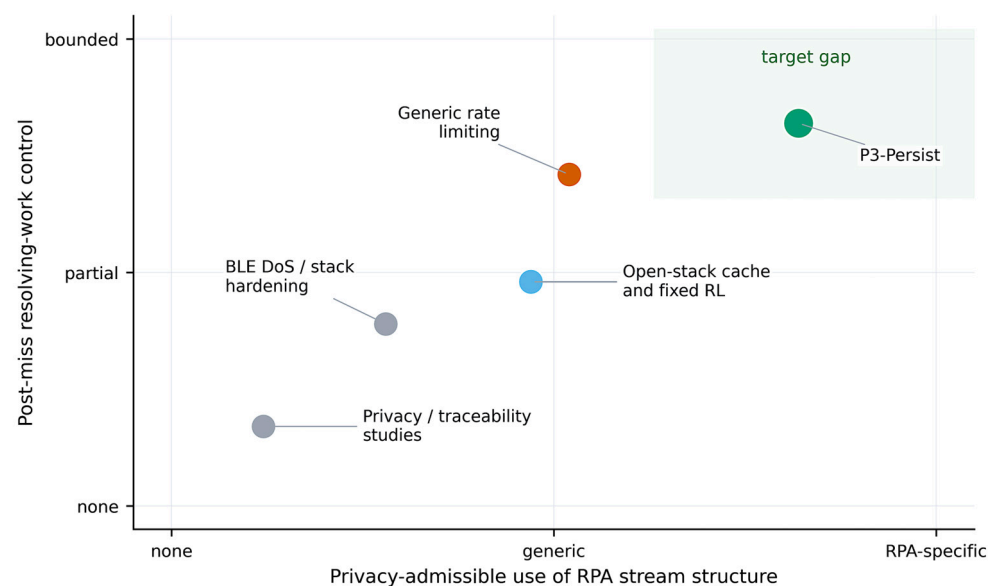


Figure 3. Related-work positioning map: P3-Persist occupies the bounded-work, no-oracle scheduling corner that prior privacy, security, cache, and generic limiter work do not jointly cover. The X-axis is the privacy-admissible use of RPA stream structure before resolving (none/generic/RPA-specific); the Y-axis is the degree of post-miss resolving-work control (none/partial/bounded).

3. Problem Formulation and Theoretical Model

3.1. Traffic Classes

The receiver observes four traffic classes. The first is legitimate bonded RPA traffic from devices whose IRKs are stored locally. The second is benign unknown RPA traffic from devices that are not bonded with the receiver. The third is non-RPA advertising

traffic that bypasses RPA resolution. The fourth is attack RPA traffic: syntactically plausible unknown RPA-like advertisements injected at high rate to amplify resolving work. Attack addresses are modeled as RPA-format random-address candidates that force the receiver into the normal resolving path, not as malformed frames or link-layer jamming.

The attacker is not assumed to break AES, recover an IRK, compromise a bonded device, or act as a Dolev–Yao protocol adversary [18]. The attacker can transmit BLE advertising packets at configurable rates and can vary the private-address-like values. The simulator uses unique attack RPA epochs as the headline stress condition because one-shot values maximize duplicate-filter evasion and cannot earn persistence reserve. To test adaptive behavior, the extended simulator also lets the attacker choose an address-pool size m , a per-address repeat factor r , and a repeat interval p , including $p < \Delta$, $p = \Delta$, and $p > \Delta$ relative to a duplicate-filter window. At the headline flood rate, the pool-index wrap dominates the per-address mapping, so the effective adaptive drivers are the pool size m and the repeat interval p while r saturates; the worst-case strategy is therefore characterized by its (m, p) values, with r still reported for exact reproducibility. The attack is effective only because the receiver may perform expensive resolving work before discovering that no local IRK matches.

3.2. Modeling Assumptions

The threat model and resolving-work model rely on the following explicit assumptions, collected here so that the scope of every later claim is visible in one place.

A1 (benign unknown traffic). Benign unknown RPA traffic from non-bonded devices is modeled conservatively as unique per event unless a scenario states otherwise. This models one-shot benign unknown traffic as a conservative source of general-budget pressure. Repeated benign unknown addresses are not covered by this assumption and may consume the persistence reserve; Section 7.2 therefore treats them as a validity boundary and a parameter-tuning concern.

A2 (legitimate RPA repetition). A legitimate bonded device emits the same over-the-air `rpa_value` while it remains within one RPA rotation epoch; this behavior follows the Bluetooth Core Specification [8], and prior advertisement-based BLE measurements provide supporting context [33]. We additionally sanity-checked the visible-address repetition premise using passive advertising traces captured by an nRF5340 DK with controller duplicate filtering disabled. Across two 1200 s controlled-on windows for each of the two smartphones, 31/34, 27/29, 35/37, and 23/26 observed that random/private-address candidate values were repeated, and the same candidate values reappeared within 60 s. These observations support visible candidate-value repetition as a real-traffic phenomenon, but they do not provide IRK-confirmed identity labels or exact RPA epoch boundaries. Sparse or irregular advertising can still reduce observed repetition in practice. Persistence-aware admission depends on observable repetition; Section 7.2 analyzes the case in which the rotation epoch is shorter than the budget window.

A3 (attack address strategy). The headline stress condition uses unique one-shot attack `rpa_values`, which maximize duplicate-filter evasion and cannot earn reserve. Repeated-address and cross-window pool strategies are evaluated separately in the sensitivity analysis.

A4 (cost abstraction). Resolving work is charged in AES-equivalent units (one unit = one IRK trial computation); no measured current, CPU cycle, or firmware timing is modeled. A host fallback after a fast-path miss is charged the conservative full-candidate cost $N-C$ regardless of IRK ordering.

A5 (capacity regime). The primary regime is $N > C$, with an nRF5x-class $C \leq 8$ resolving-list anchors; deployments with $N \leq C$ fall outside the main threat model.

3.3. Notation

Table 2 defines the notation used in the resolving-work model and in the P3 scheduler. The notation is intentionally implementation-aware: it represents controller resolving-list capacity, host IRK-search cost, receiver-local cache state, and budget-window state without introducing a pre-resolution identity predicate. Two terms are fixed here as canonical and used consistently in the rest of the paper. A bonded identity is a peer whose Identity Resolving Key is stored locally; this is the standard term throughout, in preference to incidental synonyms such as “bonded device” or “paired device.” Host fallback is the receiver-side IRK search performed over the host key store after both the positive cache and the controller resolving list miss; it is the canonical term for the expensive post-miss resolving path.

Table 2. Main notation used in the theoretical model. Italic symbols denote model variables.

Symbol	Definition
N	Number of bonded identities and locally stored IRKs.
C	Controller resolving-list capacity, with $0 \leq C \leq N$.
I	Set of bonded identities/IRKs stored by the receiver.
RL_t	Set of identities loaded into the controller resolving list at time t , with $ RL_t \leq C$.
K_t	Positive RPA cache at time t , containing only previously resolved RPA-to-identity mappings.
e_t	Advertising event observed at time t .
$rv(e_t)$	Visible over-the-air RPA value carried by event e_t .
z_t	Post-decision event class used only for evaluation labels, not a scheduler input.
E_w	Set of advertising events observed in budget window w .
$h(e_t)$	Host resolving admission indicator after cache and resolving-list misses.
$a(e_t)$	AES-equivalent attempts charged to event e_t .
B	General unresolved-work budget per window.
R	Persistence reserve per window for repeated visible RPA values.
W	Budget-window length in seconds.
k	Persistence threshold; reserve admission requires at least k prior denials for the same rv .
$D(rv)$	Receiver-local cumulative denial counter for visible value rv ; reserve usage, not this denial history, is reset by window.
u_w, r_w	General-budget and reserve counters used in window w .
q_i	Expected observation probability of bonded identity i in the idealized allocation model.
x_i	Binary resolving-list allocation variable for identity i .

3.4. Resolving-Work Model

Let N denote the number of stored bonded identities and C denote controller resolving-list capacity. When $N \leq C$, all known IRKs can be placed in the fast path. When $N > C$, only a subset can occupy the controller list. Remaining identities may still be resolved through host fallback, but host work may scale with the number and ordering of candidate

IRKs. The critical condition is therefore precise rather than rhetorical: for an nRF5x-class $C \leq 8$ resolving-list anchors, the modeled host-fallback surface begins at $N = 9$, while deployments with $N \leq C$ are outside the main problem regime. At the model level, a resolving-list hit is treated as a fast-path operation. A host miss can require multiple AES-equivalent attempts. A host match may be cheaper if the matching IRK is ordered early and more expensive if it appears late. Unknown-RPA misses are particularly costly because no match exists, so a naive host fallback can scan the full set of candidate IRKs repeatedly.

For an RPA event e_t , the receiver first checks the positive cache and the controller resolving list. The event enters host fallback only after both fast paths miss. The conservative per-event AES-equivalent cost used in this paper is

$$a(e_t) = \begin{cases} 0, & \text{if } e_t \text{ is non-RPA or hits } K_t; \\ 1, & \text{if } e_t \text{ hits } RL_t; \\ N - C, & \text{if } e_t \text{ is admitted to host fallback after } K_t \text{ and } RL_t \text{ miss;} \\ 0, & \text{if } e_t \text{ is denied or deferred by the budget policy.} \end{cases} \quad (1)$$

Equation (1) deliberately charges a full candidate-set scan after a fast-path miss. This avoids crediting P3 for unmeasured host-ordering savings. The two zero-cost cases in Equation (1) represent different fast paths: non-RPA advertisements bypass private-address resolving, whereas cache hits are already-resolved RPA values that avoid a new host IRK scan. The simulator uses z_t only after the scheduling decision to score legitimate resolution and attack-attributable amplification; it is not available to the scheduler.

For a window w with event set E_w , the unresolved host-work component is

$$H_w = \sum_{e_t \in E_w} h(e_t) \mathbb{1}\{e_t \text{ misses } K_t \text{ and } RL_t\}, \quad \Phi_{AES}(w) \leq H_w (N - C) + L_w \quad (2)$$

where L_w is the small resolving-list-hit charge from known fast-path matches. Under FullScan-Host or StaticRL without an unresolved-work bound, H_w can grow with the number of attack RPA-like events that miss the fast paths. P3 instead enforces the budget and reserve constraint:

$$u_w \leq B, \quad r_w \leq R, \quad H_w \leq B + R \quad (3)$$

with the stronger unique-address condition $H_w \leq B$ when attack values do not repeat. The corresponding attack-attributable AES-equivalent amplification bound is

$$\begin{aligned} A_{attack}(w) &\leq 60 B (N - C) / W && \text{for unique-address flooding,} \\ A_{attack}(w) &\leq 60 (B + R)(N - C) / W && \text{for arbitrary flooding.} \end{aligned} \quad (4)$$

Equation (4) is the analytical reason the reported attack work remains approximately rate-independent at high flood rates. It also states the trade-off honestly: increasing B or R can improve service for legitimate traffic, but it raises the absolute work admitted per window.

3.5. Resolving-List Capacity Allocation

Resolving-list placement is a constrained capacity-allocation problem. In an idealized one-window model, the receiver would choose binary variables x_i indicating which bonded identities are loaded into the controller resolving list:

$$\begin{aligned} (P1) \quad &= \text{maximize} \quad \sum_{i \in I} q_i x_i \\ &\text{subject to} \quad \sum_{i \in I} x_i \leq C, \\ &\quad x_i \in \{0, 1\}. \end{aligned} \quad (5)$$

The objective in (5) maximizes the expected activity covered by the fast path, equivalently minimizing expected host fallback for bonded traffic under the idealized probabilities q_i . The implemented P3 policy approximates q_i using the online counter `activity_count_i`, updated only after a successful IRK/AES-equivalent match. This substitution is a deployable simplification rather than a claim that counts are calibrated probabilities. It makes the allocation usable in a host stack: unknown misses never promote an identity into the resolving list, and no pre-resolution classifier is used.

The persistence reserve is defined by the admission indicator

$$p(e_t) = \mathbb{1}\{D(rv(e_t)) \geq k\} \cdot \mathbb{1}\{r_w < R\} \quad (6)$$

General budget admission is attempted first. If the general budget is exhausted, Equation (6) grants a reserve slot only to a visible RPA value that has reappeared. For a legitimate device whose RPA repeats during an epoch, this creates a bounded recovery path after one or more denials. For a high-amplification attacker using one-shot values, the condition remains false.

The worked example in Section 4.7 illustrates this sequence concretely: event #382 records the first denial for the repeated visible value, event #397 receives reserve admission after repetition, and events #403 and #430 become positive-cache hits after successful host resolution. These labels are used only for offline explanation and metric scoring; the scheduler does not receive legitimate/attack labels as inputs.

The theory-to-evaluation mapping is explicit. Equation (1) defines the per-event AES-equivalent cost that is summarized in Tables 14–19. Equations (2)–(4) define the unresolved-work and amplification bounds that Figure 9 and Tables 14, 15 and 17–19 test under attack rate, duration, capacity, and counter-strategy changes. Problem (P1) in Equation (5) is evaluated through resolving-list capacity and admission behavior in Figures 12 and 13, Table 19, and the ablation in Figure 14. Equation (6) is evaluated directly by the persistence comparisons in Table 17, the reserve/threshold sensitivity in Table 18, and the seed-level confidence-interval plot in Figure 17. The path-level consequences of the same model are shown in Table 21 and Figure 16, while Table 20 states the boundary that no queueing-latency or measured-current claim is made from the present simulator.

3.6. Goal and Non-Goals

The goal is to reduce attack-triggered AES-equivalent work while explicitly reporting the legitimate-deferral cost in the modeled setting. A secondary goal is to allocate resolving-list capacity to identities that are likely to benefit from the fast path. P3 must remain compatible with the privacy intent of RPA rotation: it must not provide an externally visible pre-resolution classification of unknown private addresses.

P3 has five non-goals:

- It is not a cryptographic protocol or a defense against IRK compromise.
- It is not a new BLE privacy primitive.
- It is not a general BLE intrusion-detection system.
- It is not a formal proof of untraceability.
- The measurement boundary is defined in Section 6.5.

Its optimality and bounds are stated only within the modeled privacy-admissible, bounded-work scheduling class; they are not unconditional guarantees about all real BLE stacks or traffic distributions. P3 does not attempt to stop radio jamming or prevent all denial-of-service attacks. Its scope is work scheduling in the private-address resolving pipeline.

4. Method

4.1. State Variables and Baselines

P3 is used as a method identifier rather than as an expanded acronym. In this manuscript, P3 denotes the persistence-aware admission-control scheduler family for BLE RPA resolution, and P3-Persist denotes the main evaluated instance. The design rests on three components: capacity-aware IRK admission, positive caching, and budgeted admission with a persistence reserve. The implemented simulator uses activity counters, last-seen state, positive RPA cache entries, resolving-list metadata, a bounded budget window, and an RSSI threshold only as an internal defer/skip scheduling hint. RSSI is not used to authenticate a peer, infer identity, or override cryptographic resolving. The budget window limits how many cache/RL misses can induce full host work in a time interval. Table 3 summarizes the terminology used for the P3 method family and baselines.

Table 3. Terminology for the P3 method family and baselines.

Term	Meaning in This Manuscript	Role
P3	Method identifier for the persistence-aware BLE RPA-resolving scheduler family	Design family
P3-Persist	Main evaluated P3 instance with unresolved-work budget and persistence reserve	Proposed method
P3-NoPersist	P3 ablation without the persistence reserve	Ablation
BudgetDoS	Budget-only unresolved-work limiter	Baseline
AdaptiveRateLimit	Generic fixed-window unresolved-work limiter	Baseline

Table 4 summarizes the baselines and variants used in the evaluation. The purpose of this table is to make the comparison explicit. FullScan-Host represents a complete but expensive host fallback. StaticRL represents a fixed controller list without dynamic admission. ZephyrCache isolates cache behavior inspired by practical stacks. LRU-RL, Freq-RL, and Random-RL test admission heuristics. RSSIHint and BudgetDoS isolate individual ideas. AdaptiveRateLimit tests a generic unresolved-work limiter. Oracle-Offline is a non-deployable upper bound that uses labels only for comparison. P3-Persist is the proposed full method.

Table 4. Baseline definitions.

Method	Role
FullScan-Host	Host scans all bonded IRKs; worst-case complete host fallback baseline.
StaticRL	Fixed resolving-list assignment without dynamic admission.
ZephyrCache	Cache-only practical baseline inspired by open stack behavior.
LRU-RL	Recency-based resolving-list replacement.
Freq-RL	Frequency-based resolving-list replacement.
Random-RL	Deterministic-seed random resolving-list replacement used as a low-cost admission baseline.
RSSIHint	RSSI-based defer hints without full P3 budget/admission. Implemented in code as TrustOnly; host fallback is charged conservatively in the AES metric.
BudgetDoS	Unknown-RPA budget without full P3 combination.

Table 4. Cont.

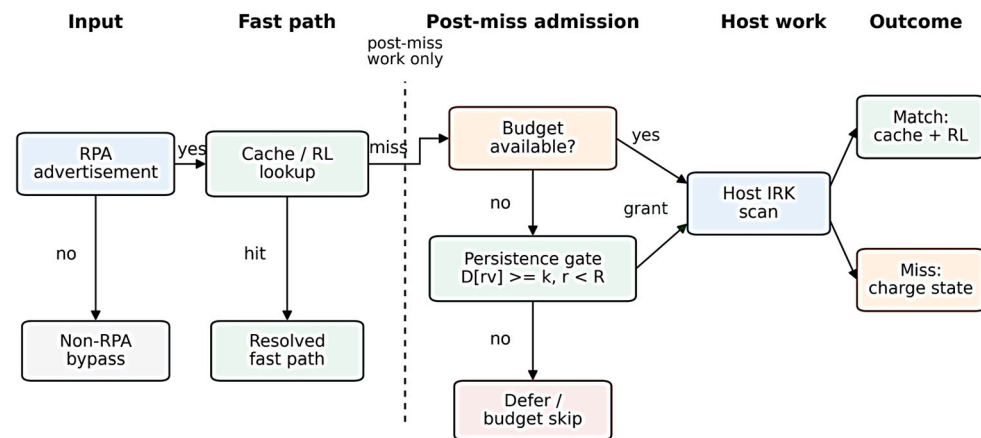
Method	Role
AdaptiveRateLimit	Generic unresolved-work limiter implemented as a fixed-window counter (threshold B host scans per W -second window, reset at each window boundary); shares the limiter core of BudgetDoS but without P3 admission/cache state.
Oracle-Offline	Non-deployable upper bound that suppresses known attack events using labels unavailable to a real receiver.
P3-Persist	Proposed full method with persistence-aware reserve after general budget exhaustion.
P3-NoPersist	Pre-persistence ablation without persistence reserve: capacity-aware resolving-list admission, positive-cache fast path, activity metadata, RSSI defer hint, and unresolved-work budget.
P3-NoCache/NoBudget/NoTrust/NoRL	Module-disabled variants for ablation.

P3-NoPersist is the pre-persistence ablation without the persistence reserve, retained so the contribution of the reserve can be measured directly. The AdaptiveRateLimit baseline is specified concretely to avoid ambiguity about its internal algorithm. It is implemented as a fixed-window counter rather than a token bucket, a leaky bucket, or a sliding-window counter: a single integer counts the host scans admitted after cache/RL misses within the current budget window of length W , the counter is reset to zero at each window boundary $\lfloor t/W \rfloor$, and any cache/RL miss is denied once the counter reaches the threshold B . It therefore shares the same amplification-bounding core as BudgetDoS but carries none of the P3 cache, capacity-aware resolving-list admission, or persistence-reserve state. This is why, as Section 6 reports, AdaptiveRateLimit and BudgetDoS suppress attack work at essentially the same level under unique flood while leaving the recovered-service gap that persistence-aware admission addresses.

4.2. Scheduling Workflow

Figure 4 shows the P3 workflow. Non-RPA traffic bypasses the resolving path. Cache hits and resolving-list hits use known fast paths. Traffic that requires host-side resolving enters the scheduler. If the general unresolved-work budget is available, host resolving proceeds over the candidate set. If the general budget is exhausted, the persistence gate checks whether the visible RPA value has reappeared enough times to earn a bounded reserve grant; otherwise the scheduler skips or defers additional expensive work for the current window. A match updates cache, activity, recency, and resolving-list admission state. A miss charges either the general budget or, when admitted by persistence, the reserve.

Algorithms 1–3 summarize the P3-Persist scheduler that the simulator executes for each incoming advertising event. Algorithm 1 keeps the BLE resolving pipeline visible, Algorithm 2 isolates the persistence-aware admission gate, and Algorithm 3 defines capacity-aware resolving-list admission after a successful match. The scheduler is intentionally conservative: it never tests whether an RPA is “unknown” before the normal resolving path. The implementation observes only packet type, cache state, resolving-list state, elapsed budget-window time, RSSI-derived defer hints, the stored IRK candidate set, and the over-the-air RPA value rv . The persistence branch is used only when the same rv has previously been denied in the current receiver state. Evaluation traffic labels are used only after a decision to score metrics such as false defer; they are not inputs to P3.



P3-Persist observes only packet type, cache/RL state, budget window, and repeated visible RPA value before host resolving.

Figure 4. P3 resolving workflow: non-RPA, cache, and resolving-list fast paths are preserved, while cache/RL misses pass through the general budget and then the persistence reserve before host fallback.

Algorithm 1. Persistence-Aware Admission Control with Persistence Reserve (P3-Persist) resolving scheduler

Input: event $e = \langle \text{addr_type}, \text{rpa_value } rv, \text{ rssi}, \text{ ts} \rangle$; resolving list RL; positive cache K; bonded IRK set I;

activity counts A; budget state Q; persistence state P; RSSI defer threshold θ .

Output: decision d, AES-equivalent cost a, and updated states.

```

1  if e.addr_type is not RPA then
2    return skip_non_rpa, 0
3  if K contains key(e) then
4    refresh K[key(e)]
5    return cache_hit, 0
6  if controller_resolve(e, RL) returns identity id then
7    K ← put(key(e), id)
8    RL, A ← Algorithm 3(id, RL, A)
9    return rl_hit, 1
10 admitted, denial_action ← Algorithm 2(e, Q, P,  $\theta$ )
11 if admitted is false then
12   return denial_action, 0
13 C_host ← candidate_set(I \ RL)
14 matched, id, a ← host_resolve(e, C_host)
15 if matched is true then
16   K ← put(key(e), id)
17   RL, A ← Algorithm 3(id, RL, A)
18 return host_scan, a
  
```

Algorithms 1–3 expose the decision order rather than introduce separate subsystems. The receiver first preserves non-adversarial fast paths: non-RPA bypass, positive cache, and controller resolving-list hits. Only cache/RL misses consume the unresolved-work budget; after that budget is exhausted, repeated visible RPA values may use a bounded persistence reserve. Successful host matches update positive cache, activity, and resolving-list state, whereas misses charge the relevant budget state without intentionally using a bonded-membership label in the initial pre-resolution decision. A cache-only optimization acts

before repeated positive resolutions; a pure rate limiter sheds work after a fixed count. P3 controls the sequence cache $\rightarrow$ RL $\rightarrow$ general budget $\rightarrow$ persistence reserve $\rightarrow$ host fallback $\rightarrow$ positive write-back/admission. The algorithmic novelty is the persistence reserve at exactly this point. It lets a legitimate device that repeats the same RPA during an epoch recover service after an initial denial, while one-shot flood addresses never accumulate $D[rv]$ and therefore cannot use the reserve.

Algorithm 2. Persistence-aware admission gate

Input: event e with visible RPA value rv and time ts ; budget state $Q = \langle B, W, w, u \rangle$; persistence state $P = \langle R, k, r, D[\cdot] \rangle$; RSSI defer threshold θ .

Output: admitted in $\{\text{true}, \text{false}\}$; denial_action in $\{\text{defer}, \text{budget_skip}\}$

```

1   $w' \leftarrow \lfloor e.ts/W \rfloor$ 
2  if  $w' \neq w$  then
3     $w \leftarrow w'$ ;  $u \leftarrow 0$ ;  $r \leftarrow 0$ 
4  if  $u < B$  then
5     $u \leftarrow u + 1$ 
6    return true, none
7  if  $D[rv] \geq k$  and  $r < R$  then
8     $r \leftarrow r + 1$ 
9    return true, none
10  $D[rv] \leftarrow D[rv] + 1$ 
11 if  $e.rssi \geq \theta$  then return false, defer
12 return false, budget_skip
```

Algorithm 3. Capacity-aware resolving-list admission

Input: resolved identity id ; resolving list RL with capacity C ; activity counts A .

Output: updated resolving list RL and activity counts A .

```

1  if  $id$  is empty then
2    return RL, A
3   $A[id] \leftarrow A[id] + 1$ 
4  if  $id$  in RL then
5    return RL, A
6  if  $|RL| < C$  then
7    insert  $id$  into RL
8    return RL, A
9   $v \leftarrow \operatorname{argmin}_{x \in RL} A[x]$ 
10 if  $A[id] \geq A[v]$  then
11   replace  $v$  with  $id$  in RL
12 return RL, A
```

The design has three invariants that are later evaluated experimentally. First, under a unique-address flood, reserve admission is unreachable because $D[rv]$ is never revisited. Second, under any repeated-address strategy, total admitted host scans per window are bounded by $B + R$. Third, resolving-list admission and cache insertion are driven only by successful resolving output (id), not by pre-resolution labels or by the persistence counter. These invariants are why P3-Persist is stronger than a presentation-level rearrangement of token-bucket limiting. The simulator implementation follows Algorithms 1–3 without using hidden oracle labels as scheduling input. Table 5 maps the manuscript-level steps to the source artifacts that generate the reported results.

Table 5. Algorithm-to-simulator mapping.

Manuscript Step	Simulator Artifact	Reproducibility Role
Traffic generation and labels	Traffic generator (generate_events)	Generates legitimate, benign unknown, non-RPA, and attack events; labels are used for metrics after decisions.
Cache hit/insert	Positive-cache state (get_cache, put_cache)	Models positive-only RPA cache behavior for ZephyrCache and P3 cache-enabled variants.
Resolving-list admission	RL admission update (maybe_update_rl)	Updates RL entries only after positive matches; activity_counts drives P3/Freq-style replacement.
Unknown-work budget	Budget gate and denial action (budget_allows, budget_denial_action)	Enforces budget_perwindow and records budget_skip/defer decisions.
Persistence-aware reserve	Reserve gate over visible RPA values (persistence_reserve_allows, rpa_value)	Grants bounded post-budget host scans to repeated over-the-air RPA values without using identity labels.
Duplicate-filter boundary	Duplicate filter (duplicate_filtered, duplicate_filterwindow_s)	Optional repeated-address boundary experiment; default 0 preserves all prior results, while enabled runs suppress unresolved same-window repeats before resolving.
Event resolution	Per-event resolver (resolve_event)	Applies cache, RL, budget, host scan, and decision accounting for each method.
Summary metrics	Summary aggregator (summarize)	Produces AES-equivalent attempts, attack amplification, legitimate resolution, false defer, and model-level delay fields.

4.3. Persistence-Aware Admission

Persistence-aware admission is the central algorithmic addition. The simulator assigns each RPA event an abstract `rpa_value` representing the over-the-air private address value visible before resolving. For a legitimate bonded device, the same `device_id` and RPA epoch produce the same `rpa_value`, reflecting the BLE behavior that an RPA remains stable during its rotation interval [8]. For default attack traffic, each event uses a unique `rpa_value`, reflecting the high-amplification strategy that avoids duplicate filtering. Benign background unknown traffic is modeled conservatively as unique unless otherwise specified. The receiver maintains a denial counter D [rv] and a small reserve R per budget window. When the general budget still has capacity, P3 behaves like the pre-persistence scheduler. When the general budget is exhausted, a value that has already been denied at least k times may use one reserve slot for a host scan. A legitimate device that repeats an RPA during the epoch can therefore recover after one or more denials and then enter the positive cache. A one-shot flood value cannot accumulate enough persistence. This is an admission mechanism, not an identity classifier: before the host scan, P3 knows only that the same address value reappeared, not which bonded identity, if any, it maps to.

The default persistence setting uses $k = 1$ and $R = 200$ per 60 s window for $N = 512$, $RL = 8$ runs. Sensitivity results show that $k = 2$ and $k = 3$ remain effective under heavy flood, with legitimate resolution 0.9086 and 0.8657, respectively, while preserving the approximately 45k amplification band. In the tested workload, R values from 50 to 400 all reach 0.9557 because the active legitimate set is below the reserve capacity; deployments

with denser legitimate traffic should tune R against observed active-device counts. Table 6 summarizes the parameters used by the scheduler and experiments.

Table 6. P3 parameters and defaults.

Symbol/Parameter	Meaning	Main/Replication Value or Range	Decision Role
N	Stored bonded identities	16–1024 in the main campaign, 512 in long-window and long-window validation runs	Host candidate-set size
$C/rl_capacity$	Controller resolving-list capacity	8, 16, 32 in the main campaign; 8, 16, 32, 64, 512 in the direct-expansion capacity check; RL = 8 headline	Fast-path capacity
$K/zephyr_cache_size$	Positive RPA cache size	64 entries	Reuse recent successful resolutions
Cache TTL	Time-based expiration	Not enabled; bounded by RPA epoch and LRU capacity	Avoids claiming an unmeasured TTL policy
$B/budget_perwindow$	Unknown-work budget	100 per window; budget sensitivity also tests larger budgets	Caps full host work after fast-path misses
$W/budget_window_s$	Budget refresh window	60 s window	Resets budget counters
$R/persist_reserve$	Persistence reserve per window	200 in the headline persistence comparison; sensitivity {50, 100, 200, 400}	Bounds post-budget host scans for repeated-address values
$k/persist_k$	Persistence threshold	1 in headline; sensitivity {1, 2, 3}	Requires reappearance before reserve admission
$D [rv]$	Denial counter for visible RPA value	Receiver-local dictionary	Tracks persistence without resolving identity
$duplicate_filterwindow_s/\Delta$	Optional duplicate-filter window	0 (off) by default; 60 s window in the repeated-address boundary experiment	Suppresses unresolved same-window repeated-address values before resolving
$activity_count_i$	Implemented admission score	Count of positive observations for identity i	Ranks RL replacement; maintained as host-order metadata but not credited in AES cost
RSSI defer threshold	Internal hint only	−78 dBm in the simulator	Chooses defer rather than skip for some budget-denied events; not an identity signal
rss_noise_db	RSSI noise level	0, 3, 6, 10 dB sensitivity	Tests robustness of RSSI-derived scheduling hints
$unique_attack_rpa$	Attack address-epoch mode	true for headline stress runs; true/false in the duplicate-filtering boundary check	Separates worst-case unique-address stress from repeated-address model sensitivity
Seeds	Random workload seeds	Main campaign: 20260530–20260603; original long-window replication: 20260610–20260612; statistical extension: 20260610–20260629	Replication and traceability

4.4. Resolving-List Admission

Resolving-list admission is a capacity-allocation problem. StaticRL keeps a fixed initial set and ignores temporal changes in device activity. LRU-RL updates entries based on recent observations. Freq-RL prioritizes frequently observed identities. The implemented P3 simulator uses a deliberately simple and reproducible score, $\text{score}_i = \text{activity_count}_i$, where activity_count_i is incremented only after a positive IRK/AES-equivalent match for identity i . When $N > C$, P3 compares the incoming matched identity with the lowest-activity resolving-list entry. Replacement is triggered only after a positive match; ties are resolved conservatively by allowing replacement when the incoming activity count is greater than or equal to the victim count. Unknown misses never cause resolving-list admission. A more general weighted activity/recency/RSSI score is a possible host policy, but it is not used as the headline implementation in this manuscript. The design principle is to improve internal resource allocation without changing protocol semantics. A bonded device that is not in the resolving list can still be resolved through host fallback when work is admitted. Admission only changes expected cost. It does not make a device trusted, authenticated, or visible to outsiders.

4.5. Host Candidate Metadata

Host fallback can be expensive if candidate IRKs are scanned in an arbitrary order. P3 maintains activity-based candidate metadata because a deployed host stack could use it to choose an order for IRK trials. In the current AES-equivalent cost model, however, a host fallback after a fast-path miss is charged the conservative full-candidate cost $N-C$ regardless of order. Candidate ordering is therefore not credited with any per-event cost reduction in the reported results. The reported P3-Persist gains come from capacity-aware resolving-list admission, the positive cache, the unresolved-work budget, and the persistence reserve, not from ordered-match savings. Crediting ordered-match cost is left to future host-timing work. This detail matters for correctness. RSSI can be noisy, spoofed, or environment-dependent. P3 does not use it as proof. It is only a scheduling hint for whether a budget-denied event is labeled as deferred or skipped in the simulator. If the correct IRK is in the candidate set and work is admitted, resolution still depends on the IRK/AES-equivalent match.

4.6. Unknown-RPA Budget

The unknown-RPA budget is the principal DoS-resilience component. A budget window allows a bounded amount of host resolving work for traffic that has missed the positive cache and resolving list. Once exhausted, additional work is skipped or deferred until the window refreshes. In the simulator, budget exhaustion produces `budget_skip` for non-prioritized traffic or defer for events above the RSSI defer threshold, and a deferred legitimate event is counted in `false_defer_legit_rate`. The budget can be configured according to expected device density and scanner capacity. It is intentionally separate from resolving-list admission because larger resolving lists do not solve repeated misses from non-bonded attackers. The evaluation includes P3-NoBudget to test this component. If the main reduction were caused by cache or resolving-list admission alone, disabling the budget would not cause a large regression. The results show the opposite: budget removal substantially increases attack amplification under flood. The budget must be interpreted carefully. It is not a classifier and it does not say that an unknown RPA is not a member of the bonded set. It only says that the receiver will not spend unbounded resolving effort on repeated misses under pressure. In normal traffic, budget pressure is low and legitimate traffic proceeds through the normal path. In attack traffic, repeated misses consume the budget and prevent unlimited full scans. The budget therefore has two security-relevant

properties. First, it is stateful at the receiver but not externally queryable as an identity predicate. An adversary may observe that a scanner under load does less work, but that observation should not reveal that a particular private address maps to a stored IRK. Second, the budget is tunable. A deployment that expects dense legitimate traffic can choose a larger budget or a shorter refresh window, while a constrained sensor gateway can choose a tighter bound. The evaluation fixes representative settings to compare methods rather than optimizing each scenario after seeing the result.

4.7. Worked Example

To make the scheduler behavior concrete, this section traces one real simulation run generated by `sim/rpa_resolution/configs/paper_walkthrough_persist.json` and stored under `sim/rpa_resolution/results/paper_walkthrough_persist/`. The run uses $N = 24$ bonded identities, resolving-list capacity $C = 4$, a budget of $B = 5$ unresolved-work units per 60 s window, persistence reserve $R = 40$, threshold $k = 1$, an attack rate of 120 RPA-like advertisements/min over a 180 s run, and seed 20260603. The example is illustrative rather than statistical, but every number in Tables 7 and 8 is directly reproducible from `events.csv`, `decisions.csv`, and `summary.csv`. Figure 5 provides the same transition as a compact event timeline before the table-level trace.

Table 7. Worked example: persistence-aware recovery of one legitimate RPA.

Event	t (ms)	Traffic Class	RPA Value	RSSI (dBm)	First Decisive Step	Action	AES	Matched
#382	126,533	Legitimate bonded	Ldev_0000:1	−65.04	Budget exhausted; first denial records $D[rv] = 1$	defer	0	no
#397	131,336	Legitimate bonded	Ldev_0000:1	−67.59	Same value reappears; $D[rv] \geq k$ grants reserve	host_scan	20	yes
#403	132,139	Legitimate bonded	Ldev_0000:1	−71.85	Positive cache after reserve resolution	cache_hit	0	yes
#430	141,211	Legitimate bonded	Ldev_0000:1	−54.23	Same-epoch repeat remains on the cache path	cache_hit	0	yes
#6	2195	Attack	A6	−83.12	Unique value denied after budget exhaustion	budget_skip	0	no
#8	3175	Attack	A12	−75.94	Unique value denied after budget exhaustion	defer	0	no

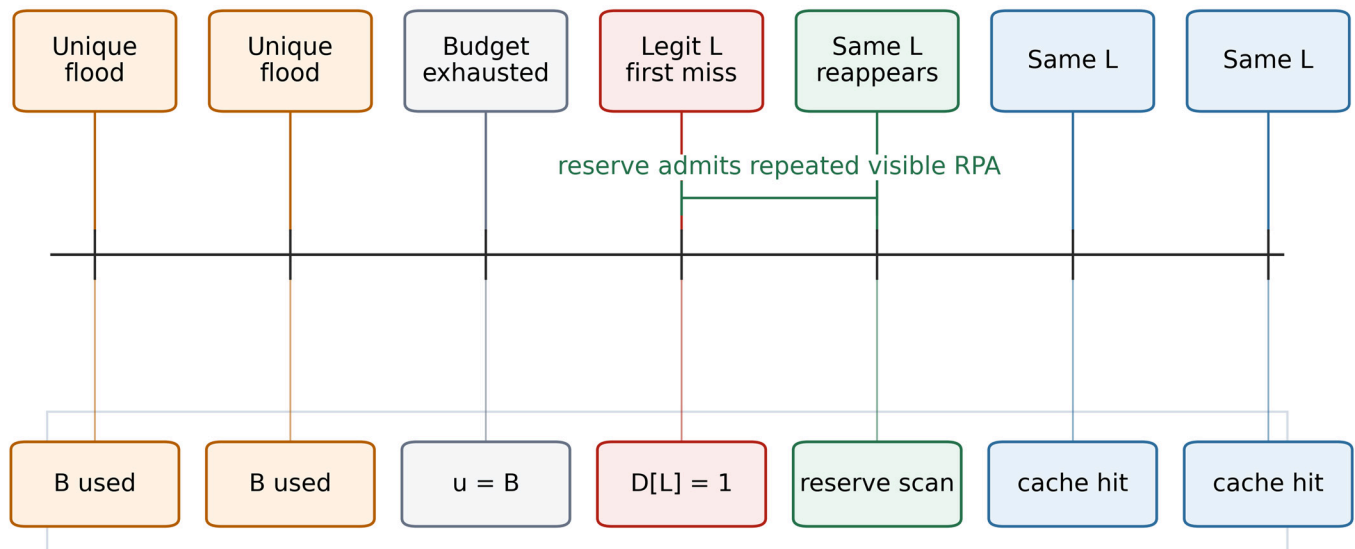
Table 8. Worked example aggregate outcome for P3-Persist.

Run	Method	Total AES	Attack Amplification	Legit Rate	False Defer
paperwalkthrough_persist	P3-Persist	487	93.4314	0.8763	0.1237

This trace shows the mechanism that the aggregate persistence rows later measure. The traffic-class column is shown only for offline explanation and metric scoring; it is not visible to Algorithm 1 or Algorithm 2 when the decision is made. Event #382 is not resolved, but its visible RPA value is recorded as a denied value. Event #397 carries the same RPA

value in the same epoch, so Algorithm 2 admits one reserve host scan; because the value is legitimate, the IRK search succeeds and Algorithm 1 writes the positive cache. Events #403 and #430 then cost AES 0. The attack examples have unique values and therefore never revisit the same $D[rv]$ state; they remain on the budget-denial path.

Incoming events



Budget / reserve state

Figure 5. Running example timeline: after budget exhaustion, a repeated legitimate RPA value earns reserve admission and then moves to the positive-cache path, while unique attack values remain denied.

The example is deliberately small so that the reserve transition is visible in a few events. It does not replace the statistical headline experiments. Its role is to connect the pseudocode to a concrete execution trace: the reserve is earned by repetition of the over-the-air value, not by pre-resolving identity membership, and once a legitimate repeat is resolved the normal positive cache absorbs later same-epoch traffic.

4.8. Analysis

This section states the guarantees of the persistence-aware scheduler as propositions over the model of Algorithms 1 and 2. Let B be the per-window budget, R the per-window persistence reserve, W the window length, k the persistence threshold ($k \geq 1$), N the number of bonded identities, and C the resolving-list capacity. A single host miss costs at most $N-C$ AES-equivalent attempts. “Amplification” denotes attack-attributable AES-equivalent work per unit time.

Proposition 0 (privacy-admissible signal characterization). *Within the threat model of Section 3 and the observation surfaces of Section 7.1, any pre-resolution admission policy whose externally observable behavior depends on whether a visible RPA belongs to the bonded set is not privacy-admissible. Therefore, before normal IRK/AES-equivalent resolving, an admissible scheduler can depend only on the observed address-value stream and non-identity hints, not on an identity-correlated membership predicate.*

Proof. Suppose a pre-resolution policy uses a signal that distinguishes, for the same externally observed address-value stream, whether a candidate RPA maps to a bonded identity. Then an observer that can replay or inject a candidate value can compare the receiver's externally visible behavior—timing, scan response, connection attempt, application-visible effect, or per-address logging surface—and infer the membership predicate. That is exactly the allowlist/membership-oracle privacy failure studied in BLE traceability work [13,18]. Hence, any privacy-admissible scheduler must be invariant to bonded membership before the normal resolving path. The remaining usable information is the visible stream itself, such as whether the same rv has appeared before, plus non-identity scheduling hints such as aggregate load or RSSI. P3 does not intentionally consume the bonded/non-bonded identity label for initial pre-resolution admission. In 120 paired simulations, identical visible traces produced identical initial decisions for all three tested methods; however, later modeled delay/defer behavior became distinguishable in some conditions after normal resolution changed cache, resolving-list, and service state. Proposition 0 is therefore a design constraint and an initial-decision invariance statement, not evidence that real end-to-end BLE behavior is side-channel free. $\square$

Proposition 1 (unique-flood amplification bound). *Under a unique-address flood, in which every attack rpa_value occurs exactly once, the persistence reserve is never granted to attack traffic, and attack-triggered host scans per window are at most B. Hence, attack amplification is at most $60B(N-C)/W$ AES-equivalent attempts per minute, independent of the flood rate.*

Proof. A reserve grant requires $D[rv] \geq k$ with $k \geq 1$ (Algorithm 2, line 7). A unique value rv is observed once; at its only arrival $D[rv] = 0 < k$, so line 7 fails and the event is denied (lines 10–12). Attack events therefore use only the general-budget branch (lines 4–6), which admits at most B host scans per window because the counter u is capped at B. Each admitted host scan costs at most N–C AES-equivalent attempts, and multiplying by 60/W converts the window bound to a per-minute rate. This bound is verified by the unique-flood row in the unique-attack experiment reported later: P3-Persist remains in the same approximately 45k AES-equivalent/min band as BudgetDoS and AdaptiveRateLimit. $\square$

Proposition 2 (arbitrary-attack amplification bound). *For any attack strategy, including repeated addresses, the total admitted host scans per window are at most B + R. Hence, attack amplification is at most $60(B + R)(N-C)/W$ AES-equivalent attempts per minute, still independent of the flood rate.*

Proof. The budget branch admits at most B host scans because line 6 is reached only while $u < B$. The reserve branch admits at most R host scans because Algorithm 2 line 7 requires $r < R$, and r is reset once per window in line 3. Their sum bounds admitted host scans by B + R; multiplying by (N–C) and by 60/W gives the per-minute cap. Table 18 reports this boundary honestly: repeated-address attack without duplicate filtering increases P3-Persist amplification to $146,163.02 \pm 374.44$ AES-equivalent/min at 10,000 attack RPAs/min, but it remains bounded rather than growing with packet rate. The 20,000/min stress point remains below the theoretical cap and has mean ratio 1.016 relative to the 10,000/min point. $\square$

Proposition 3 (legitimate-service lower bound). *Suppose at most $m \leq R$ active legitimate rpa_values are denied in a window, and each reappears within its RPA rotation epoch. Then, every such value receives a reserve host scan after at most k-denied reappearances and is absorbed by the*

positive cache thereafter. Consequently, the steady-state legitimate-resolution rate increases relative to budget-only limiting without enlarging B .

Proof. A legitimate device repeats the same rpa_value during its epoch (Bluetooth RPA rotation semantics [8]). After k denials, $D[rv] \geq k$, since at most $m \leq R$, reserve slots are needed, Algorithm 2 line 7 admits a host scan. A correct IRK match writes the positive cache (Algorithm 1, line 13), so subsequent same-epoch repetitions hit the cache at AES 0 and consume neither budget nor reserve. This bound applies only while the repeated value reappears before its RPA epoch ends and while the active repeated-legitimate set fits within the reserve; it is a modeled service-recovery condition, not a protocol-level correctness proof. This is exactly the transition illustrated in Table 7. At the aggregate level, Table 17 shows the corresponding service effect: P3-Persist raises legitimate resolution to 0.9569 under medium flood and 0.9557 under heavy flood, while BudgetDoS and AdaptiveRateLimit remain in the 0.57–0.62 range. $\square$

Proposition 4 (duplicate-filtered arbitrary-attack bound). *Let a controller or host duplicate filter with window Δ remove any subset of repeated visible-address events before they reach the resolving scheduler. Filtered events induce no host-resolving work, and every delivered event remains subject to the general budget B and persistence reserve R . Therefore, admitted host scans remain at most $B + R$ per W -second window and attack amplification remains at most $60(B + R)(N - C)/W$ AES-equivalent attempts/min.*

Proof. Duplicate filtering can only remove events before admission; it cannot add an admission path. Among the events that remain visible to Algorithms 1 and 2, the general-budget branch admits at most B host scans and the reserve branch admits at most R . The same counting argument as Proposition 2 therefore applies after filtering. This proposition does not assert that duplicate filtering collapses every repeated-address strategy. $\square$

Table 18 reports both the mitigation and its boundary. Simple high-rate repeats fall from $146,163.02 \pm 374.44$ to 504.00 ± 0.00 AES-equivalent/min with a 60 s duplicate-filter window. In contrast, the preregistered adaptive local search identifies a tied damaging region near $m = 320$ and $r = 1$: at $\Delta = 60$ s, and the highest confirmed mean is $146,196.29 \pm 380.58$ AES-equivalent/min (95% CI 146,018.17–146,374.41), with 0.6749 legitimate resolution and 0.9706 of attack events removed by duplicate filtering. The maximum seed is 147,006.72 AES-equivalent/min. These values remain below the 151,200 AES/min cap but show that a high duplicate-removal fraction does not imply low reserve work.

Design Rationale 5 (Empirical service-work trade-off within the privacy-admissible bounded-work class). *This is a design rationale rather than a formal proposition: a full game-theoretic formalization with an explicit strategy space, objective, and probability model is left to future work. Within the evaluated model, the persistence-aware reserve provides an empirically observed service-work trade-off while respecting the $B + R$ work bound and the initial pre-resolution identity-label-invariance design constraint. This study does not establish optimality over all possible privacy-admissible schedulers; other pre-resolution policies based on non-identity information remain possible subjects for future comparison.*

Argument. Proposition 2 gives the common work envelope for every bounded-work policy in the class: admitted host scans per window cannot exceed $B + R$. Proposition 0 removes identity membership and Bloom-style prefilters, and allows list-conditioned responses from the admissible signal space. The remaining pre-resolution information includes the visible address stream and non-identity scheduling hints. Under the modeled workload, legitimate bonded devices repeat the same RPA during an epoch, while a high-

amplification duplicate-filter-evasive flood prefers one-shot values. These observations motivate by allocating reserve to repeated visible values and are evaluated through the ablation and sensitivity results. They do not establish likelihood-based optimality, dominance over all alternative non-identity policies, or a complete service-work frontier. An oracle that classifies bonded membership could do better, but it lies outside the admissible class and is reported only as a non-deployable upper bound.

Propositions 1 and 2 bound unique and arbitrary attack work, Proposition 3 states the modeled repetition-dependent service condition, and Proposition 4 bounds the tested adaptive repeated-address strategies. Design Rationale 5 limits the contribution to the empirically observed privacy-admissible service-work trade-off. Figure 6 maps each claim to its evaluation evidence.

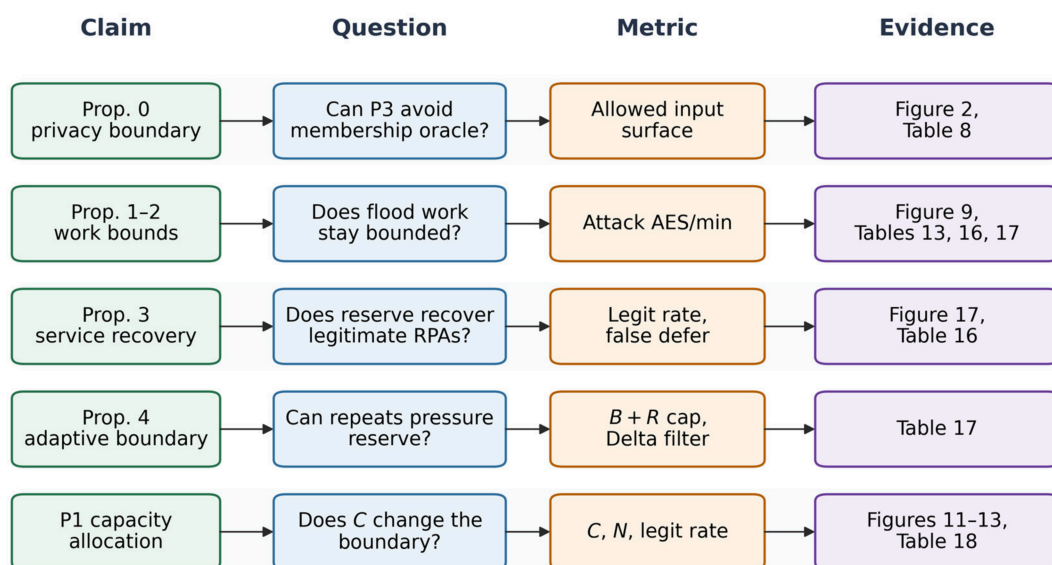


Figure 6. Proposition-to-evidence roadmap: each analytical bound is linked to the simulation table or figure that tests the corresponding work, service, or adaptive-attacker claim. Arrows read as “claim is tested by metric, and metric is evidenced by artifact”.

4.9. The Privacy-Admissible Signal Space and Why Persistence Is the Right Primitive

BudgetDoS, AdaptiveRateLimit, and P3-Persist share the same post-miss work-bounding objective. Proposition 0 excludes bonded-membership classifiers; P3-Persist therefore differs by spending bounded reserve on repeated visible values. Table 9 summarizes this mechanism-level contrast, while the corresponding evaluation section reports the statistical results.

Table 9. P3-Persist versus budget-only limiting at the same amplification target in Table 16.

Dimension	BudgetDoS/ AdaptiveRateLimit	P3-Persist	Δ That Matters
Budget placement	After cache/RL misses	After cache/RL misses	Same flood-suppression core
Post-budget admission	None; denied events are dropped by policy	Repeated rpa_value may earn reserve after k denials	New BLE-RPA-specific admission signal
Privacy input	No identity classifier	Address repetition only, no identity classifier	Supports initial-decision identity-label invariance; later modeled behavior is reported separately

Table 9. Cont.

Dimension	BudgetDoS/ AdaptiveRateLimit	P3-Persist	Δ That Matters
Medium flood, 20-seed statistical extension	AdaptiveRateLimit legit 0.6235 [0.6176, 0.6293]; BudgetDoS 0.5776 [0.5596, 0.5955]	0.9569 [0.9559, 0.9580] legit, false defer 0.0484	Same ~44k–45k amp, much higher legitimate service; paired $p < 0.01$
Heavy flood, 20-seed statistical extension	AdaptiveRateLimit legit 0.5910 [0.5861, 0.5959]; BudgetDoS 0.5663 [0.5351, 0.5975]	0.9557 [0.9547, 0.9567] legit, false defer 0.0497	Fixes the strongest-stress weakness of pre-persistence P3; paired $p < 0.01$
Attacker counter-strategy	Unique and repeated addresses are treated alike by the limiter	Unique addresses cannot earn reserve; repeated values may earn bounded reserve, and duplicate filtering may improve service without eliminating cross-window reserve pressure	Produces an explicit $B + R$ work bound and a measured service–work trade-off, not a universal repeated-address dilemma

Budget-only methods stop unlimited work but uniformly discard post-budget events. P3-Persist uses repetition to recover modeled legitimate service within the $B + R$ cap. The adaptive local refinement shows that duplicate filtering removes most events in the damaging $m = 320$, $r = 1$ region yet leaves P3 reserve work near the cap; this bounded service–work trade-off is reported in the evaluation section. ZephyrCache is a fast-path reuse baseline rather than a work-bounding policy.

Figure 7 plots this same-amplification, different-service-cost relationship for BudgetDoS and P3 variants.

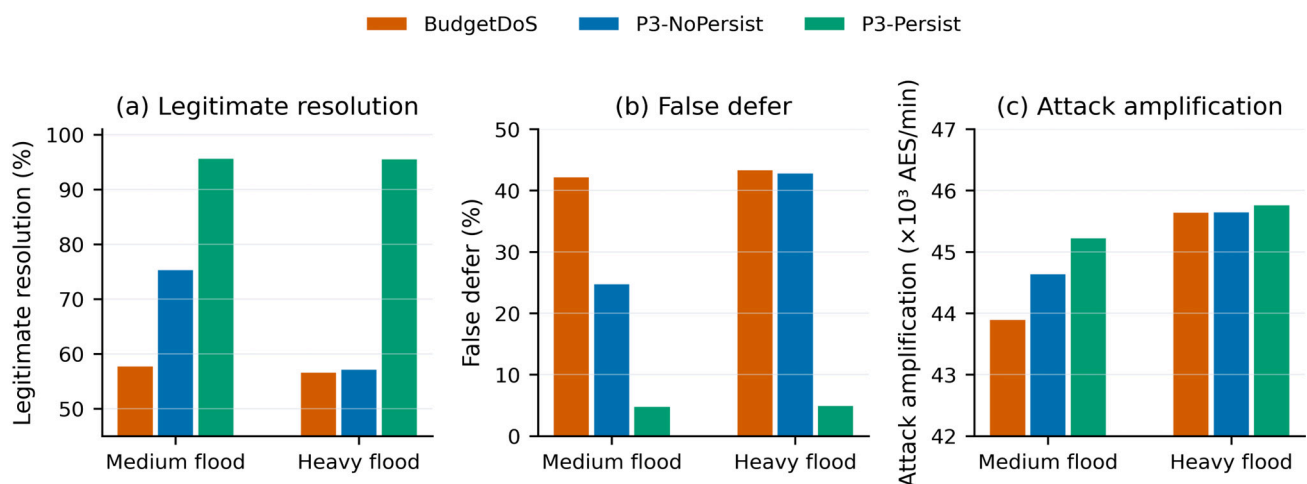


Figure 7. Same flood-suppression regime, different legitimate-traffic cost. Three panels compare BudgetDoS, P3-NoPersist, and P3-Persist under medium and heavy flood: (a) legitimate resolution rate (%), (b) false defer rate (%), and (c) attack amplification ($\times 10^3$ AES-equivalent/min). P3-Persist moves almost vertically, raising legitimate resolution while staying in the same approximately 45k AES-equivalent/min work band.

Figure 7 and Table 17 show that P3-Persist increases legitimate resolution above 0.95 while remaining in the approximately 45k AES-equivalent/min unique-flood band.

4.10. Model-to-Stack Correspondence

Table 10 maps simulator concepts to public BLE/Zephyr-NCS stack anchors without claiming firmware deployment.

Table 10. Model-to-stack correspondence.

Simulator Concept	BLE/Zephyr-NCS Correspondence	Boundary
Resolving list RL	Controller resolving list, e.g., CONFIG_BT_CTLR_RL_SIZE in Zephyr/NCS [11,21]	Capacity is modeled; closed vendor-controller timing is not.
Positive RPA cache K	Controller peer RPA cache and host per-key RPA cache [11,21]	Only positive matches are cached; unknown-miss caching is not modeled.
Host candidate set $I \setminus RL$	Host key pool searched by IRK matching, represented by Zephyr bt_keys_find_irk() code path [11]	AES-equivalent attempts model work, not measured CPU cycles.
Budget window (B, W)	Receiver-local unresolved-work control before repeated full host scans	Not a BLE protocol field and not externally queryable.
rpa_value repetition	Over-the-air RPA value within an epoch	Used only as repetition evidence; not an identity predicate.
Persistence reserve (R, k)	Receiver-local post-budget admission for repeated-address values	Not advertised to peers and not a per-address membership response.
activity_count_i	Host-side scheduling metadata derived from positive matches	Does not authenticate a device.
RSSI defer hint	Scanner-observed signal used only for internal scheduling	Not an identity predicate and not a security decision.

4.11. Complexity and Overhead

Full host scanning is $O(N)$ per miss in the number of bonded identities. Controller resolving-list hits, cache hits, budget checks, and persistence-counter checks are treated as $O(1)$ in the model. P3 adds bounded state for cache entries, activity counters, last-seen records, resolving-list metadata, RSSI hints, budget windows, and a receiver-local denial counter for recently observed RPA values. Updating the score, budget, and reserve state is lightweight relative to repeated host-side resolving scans. Table 11 summarizes complexity and model-level state; the SRAM figures follow the simulator's conservative estimator: 16 bytes per cache entry, four bytes per resolving-list metadata entry, eight bytes per bonded identity for dynamic counters/timestamps, 64 bytes for budget state, 16 bytes for RSSI hint state, and eight bytes per configured reserve slot for persistence state. The paper reports energy only as AES-equivalent work and does not report measured current.

Table 11. Complexity and model-level state.

Method	Per-Event Resolving Cost After Fast-Path Miss	Additional Modeled State
FullScan-Host	$O(N)$	None beyond stored IRKs
StaticRL	$O(N-C)$ after RL miss	Fixed RL entries
ZephyrCache	$O(1)$ cache hit, $O(N-C)$ miss	64×16 B positive cache entries
P3-NoPersist	$O(1)$ cache/RL/budget check, bounded host scan when admitted	Cache + RL metadata + per-identity counters/timestamps + budget + RSSI defer hint
P3-Persist	$O(1)$ cache/RL/budget/reserve check, bounded host scan when admitted	P3-NoPersist state + denial counters + persistence reserve state

5. Experimental Design

5.1. Research Questions

The evaluation is organized around three research questions.

RQ1 asks how bonded-device scale and resolving-list capacity affect resolving work. This establishes why scheduling is needed. If all identities fit in the controller list, capacity pressure is low. If the bonded set grows beyond capacity, the scanner must decide which identities receive fast-path treatment.

RQ2 asks how much attack-triggered AES-equivalent work P3 can reduce under RPA-flooding. This is the main result. The comparison focuses on attack amplification relative to StaticRL and other baselines.

RQ3 asks whether the unique-flood service benefit comes from generic budgeting alone or from persistence-aware admission, and how that benefit changes under adaptive repeated-address pressure. This question isolates the contribution of persistence-aware admission from generic budgeting and caching. Since practical stacks may already have cache behavior and generic work limiters are well-known, the paper must show that P3-Persist adds a measurable service benefit at the same unique-flood amplification bound, while reporting the higher $B + R$ reserve-pressure boundary separately. The results are presented in defense-first order because the attack-work bound motivates the later capacity and service-recovery analyses.

5.2. Workloads and Scripts

Figure 8 connects experiment families, generator scripts, and evidence roles. The simulator and analysis scripts are included in the reproducibility package. The main simulation campaign is generated by the matrix runner and summarized by the analysis script; long-window replication adds representative 1800 s runs through the replication runner. The reproduction audit records script versions, output paths, row counts, and the numerical values that appear in the manuscript. The figure replaces the earlier evaluation-matrix screenshot so that experiment organization is shown as a readable evidence roadmap rather than as a compressed table image.

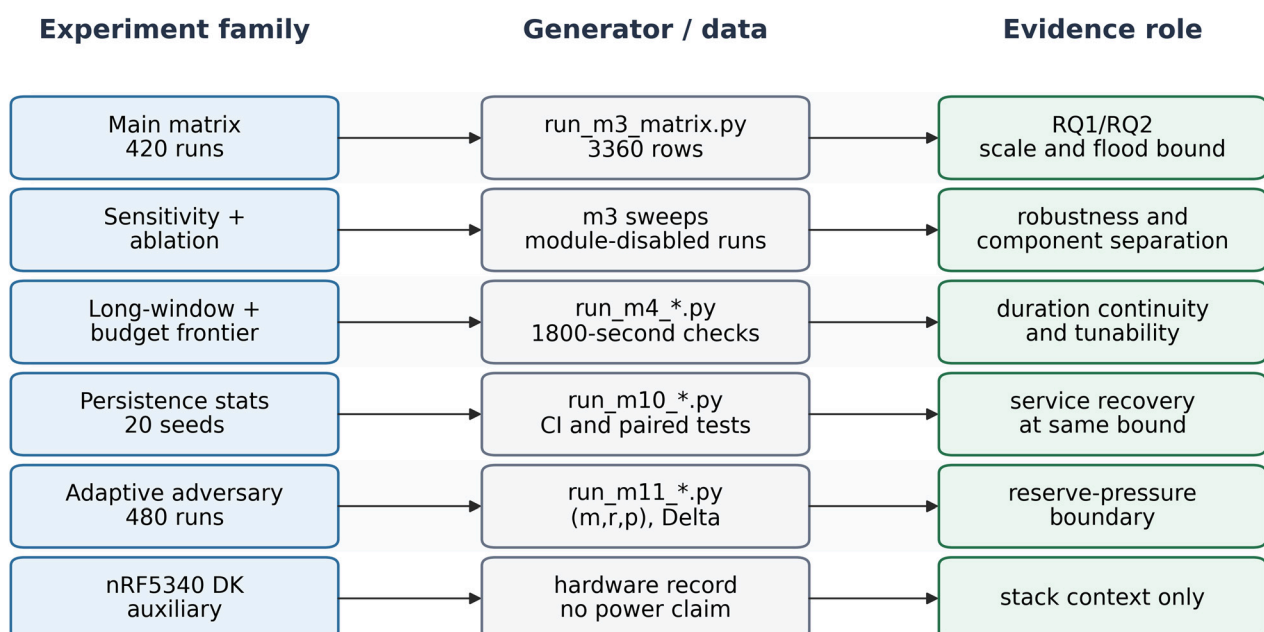


Figure 8. Experimental campaign roadmap: workload families, generator scripts, summary artifacts, and evidence roles are linked so each reported table and figure has a reproducible source.

Table 12 lists every dataset family with its generator script, run count, summary rows, and compact configuration summary; Table 13 explains why public BLE datasets do not directly support resolving-layer DoS evaluation.

Table 12. Simulated datasets and experiment artifacts.

Dataset/Artifact Family and Generator	Runs	Summary Rows	Key Configuration	Purpose and Parameter Basis
Main simulation campaign (run_m3_matrix.py)	420	3360	5 seeds; 300 s; $N = 16 \dots 1024$; $C = \{8, 16, 32\}$; attack $\{0, 100, 1000, 10,000\}/\text{min}$	RQ1/RQ2; RL = 8 anchor [11], attack feasibility [26], RSSI range [29]
Sensitivity sweeps (run_m3_matrix.py)	80	640	One-factor sweeps for background, activity skew, RPA rotation, RSSI noise	Sensitivity; RSSI noise anchored by [29]
Ablation (run_m3_ablation.sh)	15	120	P3 module-disabled variants	RQ3 module contribution
Long-window replication (run_m4_replication.py)	9	54	3 seeds; 1800 s; 300 s warmup; $N = 512$	Replication continuity
Budget sensitivity (run_m4_budget_sensitivity.py)	36	108	$B = \{100, 250, 500, 1000, 2500, 5000\}$; 1800 s; medium/heavy flood	Trade-off curve; token-bucket analogy [22]
Long-window validation (run_m6_review_closure_experiments.py)	60 configs	48	5 seeds; added baselines, capacity expansion, duplicate boundary	Boundary checks; rate limiting [22], Bloom counter-case [19]
Persistence-comparison suite (run_m8_algorithm_novelty_experiments.py)	60	135	5 seeds; P3-Persist, P3-NoPersist, reserve/threshold sensitivity, attacker dilemma, duplicate-filter boundary	Persistence-aware evidence; RPA epoch semantics [8], rate limiting [22]
Statistical extension (run_m10_statistical_extension.py)	280	580	20 seeds; 1800 s; headline persistence, reserve/threshold sensitivity, attacker dilemma, 10,000/20,000 min rate-independence check	Paired tests and Proposition 2 stress validation
Adaptive-adversary sweep (run_m11_adaptive_adversary.py)	480	960	20 seeds; 1800 s; 12 attack strategies over unique, legacy repeated, and representative (m, r, p) pools with $\Delta = \{0, 60\}$	Proposition 4 boundary validation; reports the worst swept reserve-pressure point
Paired modeled-observable privacy suite (run_m12_privacy_sidechannel.py)	240 variants	720	120 visible-trace-matched pairs; $3 \text{ loads} \times \Delta = \{0, 60\} \times 20 \text{ seeds}$; P3-Persist, P3-NoPersist, BudgetDoS	Initial-decision identity-label invariance audit and modeled external/internal distinguishability boundary

Table 12. Cont.

Dataset/Artifact Family and Generator	Runs	Summary Rows	Key Configuration	Purpose and Parameter Basis
Joint robustness suite (run_m13_joint_robustness.py)	600	1800	epoch = {1,5,15 min} × observation loss = {0, 0.2, 0.4} × benign density = {100, 1000, 5000}/min; 20 seeds; three controls	Offered/observed service, false defer, work bound, and repeated-benign boundary
Adaptive local refinement (run_m14_adaptive_refinement.py)	1280	1480	168-strategy 5-seed screen plus 11-strategy $\Delta = \{0, 60\}$, 20-seed confirmation; BudgetDoS paired on top five	Dense local best-response search, r -effect audit, tied-region confirmation, and cap-distance audit
Worked-example traces (paperwalkthrough*.json + rpa_sim.py)	2	577 events × 3 methods; 591 events × 1 method	seed 20260603; pre-persistence trace plus P3-Persist trace with $N = 24$, $C = 4$, $B = 5$, $R = 40$, $k = 1$; traces enabled	Running example and persistence trigger trace

Table 13. Public BLE datasets considered and why they do not directly support resolving-layer DoS evaluation.

Public Dataset	Content	Why Not Directly Usable Here
MIT Lincoln Laboratory BLE exposure-notification data collection report [35]	RSSI under controlled proximity scenarios	Proximity RSSI only; no IRK/resolving-list state, no unknown-RPA flooding, no host resolving-work labels [36]
Indoor BLE localization dataset [37]	RSSI fingerprints from devices to fixed anchors	Localization labels; does not expose private-address resolution or AES-equivalent matching cost
BLEBeacon dataset [38]	Beacon advertisement RSSI, timestamps, IDs	Occupancy/tracking traces; no resolving-list capacity pressure or attacker-injected RPA work [39]
BLE RSSI variability measurements [40]	RSSI vs. distance, orientation, and interference [41]	Used here only to anchor RSSI parameter ranges; not a resolution-workload dataset

All datasets are produced by `sim/rpa_resolution/src/rpa_sim.py` and the listed scripts. Each run is deterministic given its seed, so any row can be regenerated exactly. One summary row corresponds to one method-by-seed-by-configuration aggregate emitted by the simulator summary stage, not to an independently sampled real deployment. Energy figures are AES-equivalent estimates, not measured current. The parameter-basis citations justify the chosen ranges; they do not imply that the data were collected from those sources.

Existing public BLE datasets provide RSSI, proximity, localization, or delivery measurements but not resolving-list occupancy, host IRK-search work, or injected unknown-RPA load [35–38]. The deterministic simulator is therefore the primary policy-comparison instrument, while public measurements and the controlled NT-03 trace provide parameter and repetition context. Table 12 records every experiment family, run/row count, seed basis, and generator so each reported result remains traceable without implying deployment coverage. The traffic-rate settings are chosen to separate ordinary capacity pressure from

flood behavior. A rate of 100 attack RPAs/min is a low stress point, with approximately 1.7 attack RPA-like advertisements/s. A rate of 1000 attack RPAs/min is the medium-flood headline point, approximately 16.7/s. A rate of 10,000 attack RPAs/min is a heavy upper stress-test, approximately 166.7/s. These rates are below the raw BLE advertising-channel packet capacity but may require a high-rate test transmitter, multiple coordinated transmitters, or a trace-replay stress harness. The heavy setting is not presented as a typical everyday deployment. Its purpose is to expose whether resolving work grows without bound or remains controlled near the configured budget.

5.3. Randomness and Replication

The main campaign uses seeds 20260530, 20260531, 20260601, 20260602, and 20260603. For each attack-rate point in Table 14, the RL = 8 headline aggregates seven bonded-device scales and five seeds, so each mean is based on $n = 35$ configuration/seed samples rather than a sample of real deployments. Sensitivity and ablation use the same five-seed family. The original long-window replication uses seeds 20260610, 20260611, and 20260612; each scenario/method row in Table 15 is an $n = 3$ long-window sanity check. The statistical persistence extension uses seeds 20260610–20260629; rows in the persistence sensitivity tables are reported as $n = 20$ unless the table states a deterministic boundary. Metrics are computed after excluding each run’s warmup prefix. The main campaign in Table 14 uses 300 s runs with a 60 s warmup, so its measurement window is 240 s. The long-window replication, persistence headline/dilemma suites, and statistical extension use 1800 s runs with a 300 s warmup, so their measurement window is 1500 s. Amplification denominators use the post-warmup measurement duration, and legitimate-resolution and false-defer rates count only post-warmup RPA events. The paper reports means, standard deviations, and Student-t 95% confidence intervals for the headline persistence comparisons. This remains a deterministic simulation study: the intervals summarize seed-to-seed variation under the configured workload family, not inference over an empirically sampled population of real deployments. The supplementary CSVs retain run identifiers, seeds, durations, warmup settings, measurement windows, event rates, duplicate-filter settings, and method summaries so that a reader can recompute standard deviations, confidence intervals, or seed-level distributions. The manuscript therefore uses wording such as “observed reduction in the simulated matrix” and avoids broader population-level significance claims.

Table 14. Main reduction numbers (RL = 8 headline; attack-amplification metric; mean $\pm$ SD over seven bonded-device scales and five seeds, $n = 35$ per attack-rate row).

Attack RPAs/min	Traffic Interpretation	StaticRL Mean $\pm$ SD	P3-NoPersist Mean $\pm$ SD	Reduction
0	No injected attack	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.0%
100	Low stress	28,602.51 $\pm$ 35,607.16	12,002.80 $\pm$ 13,736.49	58.0%
1000	Medium flood, 16.7/s	284,074.69 $\pm$ 348,379.74	24,597.14 $\pm$ 29,519.86	91.3%
10,000	Heavy upper stress-test, 166.7/s	2,818,840.11 $\pm$ 3,441,379.85	27,758.97 $\pm$ 33,787.36	99.0%

Table 15. Long-window validation summary for $N = 512$ and RL = 8 (1800 s runs; 300 s warmup; $n = 3$ seed means). The complete method-by-scenario table is provided as Supplementary Table S1.

Scenario	Method	Attack/Min	Attack Amplification	Legit Rate	False Defer
heavy_flood	BudgetDoS	10,000	45,192.00	0.5233	0.4767
heavy_flood	P3-NoPersist	10,000	45,198.72	0.5602	0.4403

Table 15. Cont.

Scenario	Method	Attack/Min	Attack Amplification	Legit Rate	False Defer
medium_flood	BudgetDoS	1000	43,874.88	0.5809	0.4191
medium_flood	P3-NoPersist	1000	44,593.92	0.7554	0.2459
normal_capacity	BudgetDoS	0	0.00	0.8814	0.1186
normal_capacity	P3-NoPersist	0	0.00	0.9855	0.0234

5.4. Metrics

The primary metric is AES-equivalent resolving attempts. Attack amplification is measured as resolving work attributable to attack traffic. Reduction is reported relative to StaticRL for the main attack-rate comparison. The evaluation also reports legitimate resolution rate, false defer rate, budget-defer cost, SRAM reporting policy, and energy reporting policy. Energy reporting is limited to AES-equivalent operation estimates; the manuscript does not report measured current, battery-life improvement, or calibrated power data. The two service-side rates are defined as follows. Let E_{legit} be the set of post-warmup legitimate bonded RPA events, and let $\text{resolved}_{\text{legit}}$ be those whose scheduling decision produced a successful IRK match. Let $\text{deferred}_{\text{legit}}$ be the post-warmup legitimate decisions whose action is defer. The legitimate resolution rate and the false-defer rate are

$$\begin{aligned} \text{legit}_{\text{rate}} &= |\text{resolved}_{\text{legit}}| / |E_{\text{legit}}|, \\ \text{false}_{\text{defer}} &= |\text{deferred}_{\text{legit}}| / |D_{\text{legit}}|. \end{aligned} \quad (7)$$

where D_{legit} is the set of post-warmup legitimate simulator decisions. The legitimate-resolution denominator counts legitimate RPA events, whereas the false-defer denominator counts all post-warmup legitimate decisions generated by the simulator, matching the current 'rpa_sim.py' fields 'legit_resolution_rate' and 'false_defer_legit_rate'. Controller resolving-list hits and positive-cache hits remain in the false-defer denominator when they produce simulator 'Decision' records; the metric therefore reports the share of modeled legitimate decisions that were deferred, not only the share among events that reached the post-miss admission gate. Both rates are computed only over post-warmup traffic, consistent with the measurement window defined in Section 5.3.

Correctness-related metrics are model-bounded. In the persistence flood scenarios, P3-Persist improves legitimate resolution to 0.9569 under medium flood and 0.9557 under heavy flood while preserving the same bounded attack-amplification regime. These values do not prove deployment correctness or failure; they define the modeled cost and benefit of persistence-aware load shedding. The current simulator does not implement a realistic event-queue latency model. Its delay field is therefore not used as a latency result.

5.5. Stack-Context Auxiliary Record

The nRF5340 DK material provides scanner-context and visible-address-repetition evidence only; Section 6.6 defines its measurement boundary.

6. Results

6.1. RQ2: RPA-Flooding Defense

The first flooding-defense result is that bounded unresolved-work admission is necessary under attack flooding. Figure 9 summarizes the heavy-flood trade-off on the main 20-seed runs at 10,000 attack RPAs/min. The left panel shows that BudgetDoS, AdaptiveRateLimit, P3-NoPersist, and P3-Persist all remain in the same bounded attack-work band,

far below the unbounded Random-RL reference. The right panel shows why P3-Persist is the main mechanism while preserving the same bounded amplification regime as budget-only methods: it preserves the bound while recovering legitimate service that budget-only limiting and P3-NoPersist drop. The earlier rate-sweep reduction numbers are retained in Table 14.

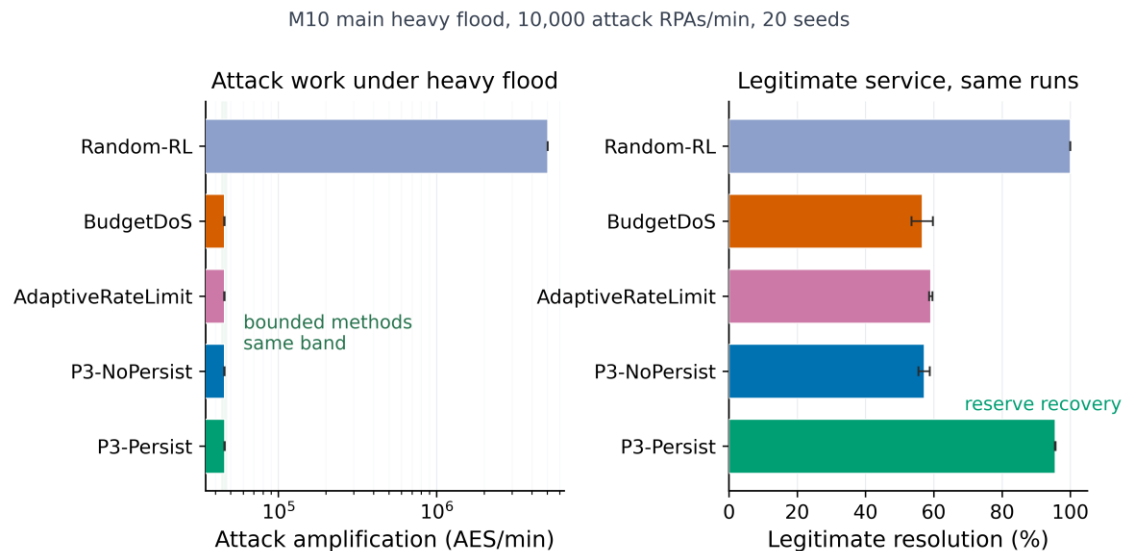


Figure 9. Heavy-flood attack-work and service trade-off at 10,000 attack RPAs/min: the left panel reports attack-triggered AES-equivalent amplification on a log scale, while the right panel reports legitimate resolution on the same main statistical-extension runs. P3-Persist remains in the bounded-work band while recovering legitimate service. Source artifacts: ‘run_m10_statistical_extension.py’, the main heavy-flood result family, and the corresponding figure-source CSVs listed in Table 12.

Table 14 shows flood-rate-dependent StaticRL work and bounded budgeted work; its large SDs mainly reflect pooling $N = 16 \dots 1024$. Tables 15 and 17 provide fixed- $N = 512$ long-window and 20-seed comparisons. The complete 18-row long-window result is provided in Supplementary Table S1, while the six-row Table 15 summary retains the medium-, heavy-, and normal-capacity continuity checks.

Table 15 also shows why persistence-aware admission is needed. Under long-window medium flood, pre-persistence P3-NoPersist resolves 0.7554 of legitimate RPA events and false-defers 0.2459. Under long-window heavy flood, the corresponding values are 0.5602 and 0.4403. These rows bound attack work but pay too much legitimate-service cost. P3-Persist is introduced precisely to recover that service without reopening unbounded host scans. Budget sensitivity makes the tunability explicit. Table 16 shows that larger budgets improve legitimate resolution but reduce attack-work reduction. Under medium flood, a budget of 500 per 60 s window raises legitimate resolution above 0.93 while retaining a 54.9% attack-work reduction. Under heavy flood, a budget of 5000 reaches a similar legitimate-resolution level but reduces attack-work reduction to 54.6%.

Figure 10 shows the expected budget frontier: larger budgets improve legitimate service but reduce attack-work suppression, with diminishing service gains near saturation.

Table 17 isolates persistence from generic limiting and caching. Random-RL preserves service without bounding attack work; BudgetDoS, AdaptiveRateLimit, and P3-NoPersist remain near the 44 k–46 k AES-equivalent/min band but lose substantial legitimate service. At the same unique-flood work band, P3-Persist reaches 0.9569 medium-flood and 0.9557 heavy-flood legitimate resolution.

Table 16. Budget sensitivity trade-off ($N = 512$, $RL = 8$, 1800 s medium/heavy flood scenarios; metric rows are $n = 3$ seed means per budget/scenario pair).

Scenario	Budget/Window	Attack/Min	Reduction vs. StaticRL	Legit Rate	False Defer
medium_flood	100	1000	91.1%	0.7554	0.2459
medium_flood	250	1000	77.5%	0.8707	0.1326
medium_flood	500	1000	54.9%	0.9365	0.0683
medium_flood	1000	1000	9.7%	0.9896	0.0171
medium_flood	2500	1000	0.0%	1.0000	0.0072
medium_flood	5000	1000	0.0%	1.0000	0.0072
heavy_flood	100	10,000	99.1%	0.5602	0.4403
heavy_flood	250	10,000	97.7%	0.6392	0.3617
heavy_flood	500	10,000	95.5%	0.7089	0.2929
heavy_flood	1000	10,000	90.9%	0.7708	0.2317
heavy_flood	2500	10,000	77.3%	0.8752	0.1277
heavy_flood	5000	10,000	54.6%	0.9403	0.0642

Figure 10 visualizes the operating frontier described by Table 16.

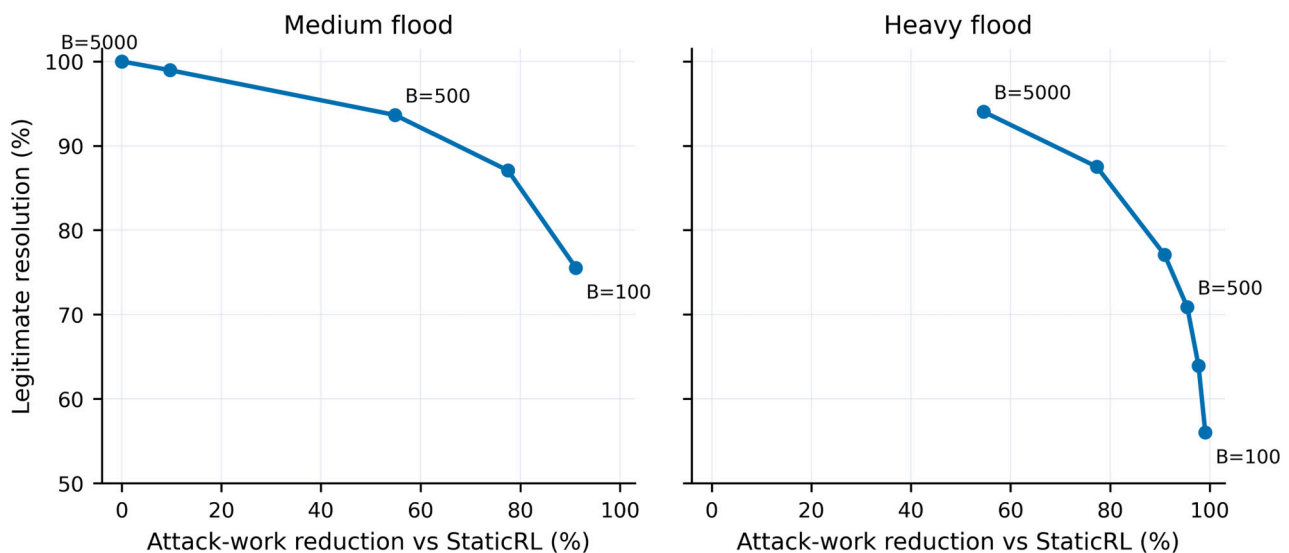


Figure 10. Budget tuning frontier: the x-axis varies the unresolved-work budget per window and the y-axis reports attack-work reduction and legitimate-resolution trade-offs under medium and heavy flood.

Table 17. Persistence baselines and upper bound ($N = 512$, $RL = 8$, 1800 s medium/heavy flood scenarios; statistical rows use $n = 20$ seeds unless marked as an oracle or deterministic boundary). The pre-persistence P3 path is reported once as P3-NoPersist to avoid duplicate equal-by-construction rows.

Scenario	Method	Attack Amplification Mean $\pm$ SD [95% CI]	Legit Rate Mean [95% CI]	False Defer Mean [95% CI]
medium_flood	Random-RL	504,557.42 $\pm$ 3220.32 [503,050.27, 506,064.58]	1.0000 [1.0000, 1.0000]	0.0000 [0.0000, 0.0000]
medium_flood	AdaptiveRateLimit	44,059.68 $\pm$ 314.67 [43,912.41, 44,206.95]	0.6235 [0.6176, 0.6293]	0.3765 [0.3707, 0.3824]
medium_flood	BudgetDoS	43,898.40 $\pm$ 301.05 [43,757.50, 44,039.30]	0.5776 [0.5596, 0.5955]	0.4224 [0.4045, 0.4404]

Table 17. Cont.

Scenario	Method	Attack Amplification Mean $\pm$ SD [95% CI]	Legit Rate Mean [95% CI]	False Defer Mean [95% CI]
medium_flood	P3-NoPersist	44,641.30 $\pm$ 301.53 [44,500.17, 44,782.42]	0.7536 [0.7467, 0.7605]	0.2481 [0.2413, 0.2549]
medium_flood	P3-Persist	45,227.95 $\pm$ 280.60 [45,096.63, 45,359.28]	0.9569 [0.9559, 0.9580]	0.0484 [0.0470, 0.0498]
medium_flood	Oracle-Offline	0.00 $\pm$ 0.00 [0.00, 0.00]	1.0000 [1.0000, 1.0000]	0.0000 [0.0000, 0.0000]
heavy_flood	Random-RL	5,038,527.31 $\pm$ 9489.79 [5,034,085.95, 5,042,968.67]	1.0000 [1.0000, 1.0000]	0.0000 [0.0000, 0.0000]
heavy_flood	AdaptiveRateLimit	45,658.37 $\pm$ 338.96 [45,499.73, 45,817.01]	0.5910 [0.5861, 0.5959]	0.4090 [0.4041, 0.4139]
heavy_flood	BudgetDoS	45,648.29 $\pm$ 350.53 [45,484.23, 45,812.34]	0.5663 [0.5351, 0.5975]	0.4337 [0.4025, 0.4649]
heavy_flood	P3-NoPersist	45,650.30 $\pm$ 342.78 [45,489.88, 45,810.73]	0.5718 [0.5550, 0.5886]	0.4284 [0.4116, 0.4452]
heavy_flood	P3-Persist	45,765.22 $\pm$ 339.36 [45,606.39, 45,924.04]	0.9557 [0.9547, 0.9567]	0.0497 [0.0487, 0.0508]
heavy_flood	Oracle-Offline	0.00 $\pm$ 0.00 [0.00, 0.00]	1.0000 [1.0000, 1.0000]	0.0000 [0.0000, 0.0000]

Table 18 separates reserve sizing, threshold sensitivity, and attacker counter-strategy behavior.

Table 18. Persistence-reserve sensitivity and attacker-strategy boundaries for $N = 512$ and $RL = 8$ (1800 s scenarios; statistical and confirmation rows use $n = 20$ seeds unless explicitly identified as the five-seed adaptive-refinement screen). For the headline setting $B = 100$, $R = 200$, $W = 60$, and $C = 8$, the arbitrary-flood cap is 60 $(B + R)(N - C) / W = 151,200$ AES-equivalent attempts/min.

Experiment	Setting	Attack Amplification Mean $\pm$ SD	Legit Rate	False Defer	Interpretation
Reserve sensitivity	$R = 50$, $k = 1$, heavy unique flood	45,765.22 $\pm$ 339.36	0.9557	0.0497	Active set fits within even small reserve in this workload
Reserve sensitivity	$R = 100$, $k = 1$, heavy unique flood	45,765.22 $\pm$ 339.36	0.9557	0.0497	Same bound and service
Reserve sensitivity	$R = 200$, $k = 1$, heavy unique flood	45,765.22 $\pm$ 339.36	0.9557	0.0497	Headline setting
Reserve sensitivity	$R = 400$, $k = 1$, heavy unique flood	45,765.22 $\pm$ 339.36	0.9557	0.0497	No extra benefit once active repeats are covered
Threshold sensitivity	$R = 200$, $k = 2$, heavy unique flood	45,754.13 $\pm$ 344.90	0.9086	0.0956	More conservative threshold remains above all baselines

Table 18. Cont.

Experiment	Setting	Attack Amplification Mean $\pm$ SD	Legit Rate	False Defer	Interpretation
Threshold sensitivity	$R = 200, k = 3$, heavy unique flood	$45,745.06 \pm 340.87$	0.8657	0.1369	Higher threshold trades service for stricter persistence
Attacker dilemma	unique-address attack, $R = 200, k = 1$	$45,765.22 \pm 339.36$	0.9557	0.0497	Unique flood cannot earn reserve
Attacker dilemma	repeated-address attack, duplicate filter OFF, $R = 200, k = 1, 10,000/\text{min}$	$146,163.02 \pm 374.44$	0.6411	0.3595	Reserve can be pressured, but work remains bounded by $B + R$
Attacker dilemma	repeated-address attack, duplicate filter OFF, $R = 200, k = 1, 20,000/\text{min}$	$148,493.52 \pm 209.70$	0.5924	0.4078	Mean remains below the 151,200 AES/min cap; 20 k/10 k seed-paired ratio mean is 1.016
Attacker dilemma	repeated-address attack, duplicate filter ON (60 s Δ), $R = 200, k = 1$	504.00 ± 0.00	0.9433	0.0621	Duplicate filtering removes high-rate repeats before reserve pressure
Adaptive sweep	Earlier coarse-grid reference, duplicate filter ON ($m = 256, r = 1, p = 120$ s)	$128,962.51 \pm 49.59$	0.9286	0.0767	Reproduced historical point; rank 82/168 in the denser adaptive-refinement screen
Adaptive refinement	Confirmed tied-region representative, duplicate filter ON ($m = 320, r = 1, p = 75$ s)	$146,196.29 \pm 380.58$	0.6749	0.3256	Highest confirmed mean; 96.69% of cap; 95% CI 146,018.17–146,374.41; duplicate-filtered attack fraction 0.9706
Adaptive refinement	Same confirmed tied-region strategy under BudgetDoS, duplicate filter ON	$45,644.26 \pm 351.86$	0.0788	0.9212	Lower work but sheds almost all legitimate traffic; paired comparator for the safety–service trade-off

Table 18 characterizes reserve sensitivity and attacker-induced reserve pressure. First, the reserve does not require fine tuning in the tested $N = 512, \text{RL} = 8$ workload: $R = 50$ already covers the active repeated legitimate values, and larger R only increases modeled state. Second, an attacker can try to repeat addresses to earn reserve, but the outcome depends on repetition timing. With duplicate filtering disabled, the repeated-address row is reported as a boundary condition rather than hidden; it is still bounded by $B + R$ host scans per window and remains rate-independent within the tested 10,000–20,000/min range. The observed repeated-address and adaptive-pool results are interpreted against the explicit arbitrary-flood cap stated in Table 18 caption. With a 60 s duplicate-filter window

enabled, simple high-rate repeats are removed before reaching the resolving scheduler, reducing attack amplification to 504.00 AES-equivalent/min while preserving legitimate resolution at 0.9433. The earlier coarse adaptive-adversary campaign comprises 480 simulator executions spanning 12 address strategies, 20 seeds, and duplicate-filter windows of 0 and 60 s. Each execution produces two policy-result rows, one for BudgetDoS and one for P3-Persist, yielding 960 summary rows. The strategies vary address-pool size m , per-address repeat factor r , and repeat interval p , including intervals below, equal to, and above the duplicate-filter window. A separate statistical extension compares 10,000/min and 20,000/min to test sending-rate sensitivity. The adaptive local refinement independently crosses $m \in \{128, 160, 192, 224, 256, 320, 384, 512\}$, $p \in \{75, 90, 105, 120, 135, 150, 180\}$ s, and $r \in \{1, 2, 4\}$ in a 168-strategy screen. The historical $m = 256$, $r = 1$, $p = 120$ s point ranks 82/168, while the damaging local region shifts to $m = 320$ and $r = 1$. The five preregistered top strategies were confirmed with 20 seeds at $\Delta = \{0, 60\}$ s, and their confidence intervals overlap. At $\Delta = 60$ s, the highest confirmed P3-Persist mean is 146,196.29 AES-equivalent/min, 96.69% of the 151,200 cap, and the maximum seed is 147,006.72. The result remains within the modeled $B + R$ envelope but leaves limited headroom, and the screen does not support the earlier r -saturation hypothesis.

6.2. RQ1: Resolving-List Capacity and Bonded Scale

When the bonded-device set is small enough to fit in the resolving list, the fast path absorbs most known traffic. As N grows beyond C , the list becomes a scarce resource and host fallback becomes more important. Figure 11 shows AES-equivalent attempts as bonded-device scale changes. Figure 12 shows the effect of resolving-list capacity, now including the P3-Persist main method. Larger resolving lists reduce pressure, but capacity alone does not address repeated misses from unknown-RPA flooding unless the list is large enough to absorb all relevant identities and avoid fallback.

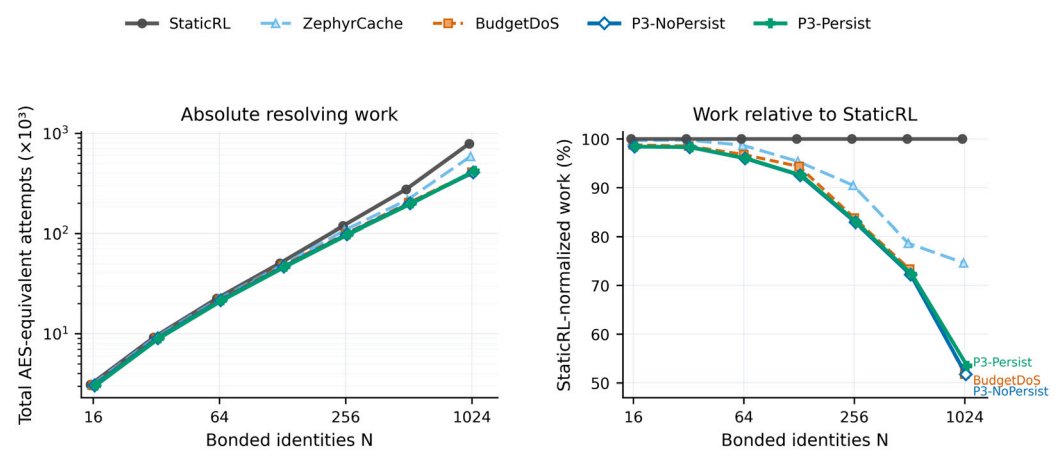


Figure 11. Bonded-device scale versus resolving work: the left panel reports total AES-equivalent attempts and the right panel reports StaticRL-normalized work, including the matched P3-Persist supplemental curve and showing where $N > C$ creates host-fallback pressure.

The direct expansion check makes this boundary explicit. At $N = 512$ and 1000 attack RPAs/min, increasing C from eight to 64 improves P3-NoPersist legitimate resolution from 0.7662 to 0.9280 and lowers false defer from 0.2355 to 0.0725, while attack amplification remains bounded near 40k–45k AES-equivalent attempts/min. At $C = N = 512$, both StaticRL and the no-reserve policy have zero attack amplification in the model because every bonded identity fits in the fast path and unknown misses do not trigger host fallback. Thus, direct expansion is a theoretical substitute when C can reach N ; the paper's practical

motivation is the realistic low-capacity regime, such as $RL = 8$ nRF-class controllers, where that expansion is not available.

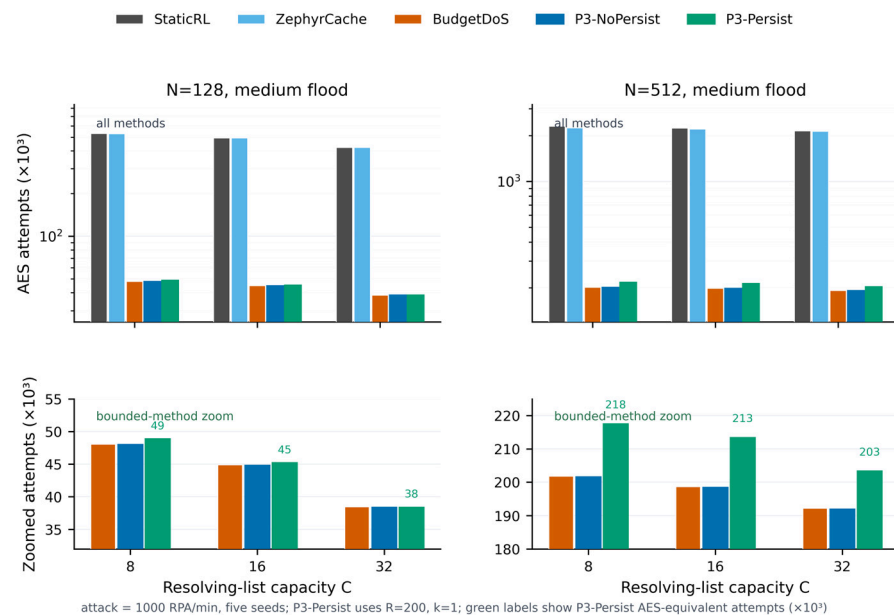


Figure 12. Resolving-list capacity sensitivity: the top panels show all methods on the shared AES-equivalent scale, and the bottom panels zoom into the bounded schedulers to separate BudgetDoS, P3-NoPersist, and the P3-Persist main method.

This result should be interpreted together with the deployment boundary in Section 1. The paper does not require the empirical claim that most BLE centrals have large bond tables. It requires the architectural condition $N > C$, which is concrete and measurable: the Bluetooth controller command set exposes resolving-list capacity, while host configuration and retained bonds determine N [8,11,21,39]. Zephyr/nRF-class targets illustrate this boundary once retained bonded identities exceed the configured controller resolving-list capacity; platforms whose host bond limit is below or equal to the controller list, or deployments that are beacon-only and never resolve bonded RPAs, fall outside the main threat model. Table 19 gives the direct resolving-list expansion check behind this boundary.

Table 19. Direct resolving-list expansion check.

C	StaticRL Attack Amp	No-Reserve Attack Amp	No-Reserve Legit Rate	No-Reserve False Defer
8	504,181.44	44,537.47	0.7662	0.2355
16	496,178.56	44,005.12	0.8020	0.1996
32	480,172.80	42,862.08	0.8560	0.1453
64	448,161.28	40,384.51	0.9280	0.0725
512	0.00	0.00	1.0000	0.0000

Figure 13 shows the corresponding capacity-expansion boundary.

Figure 13 evaluates whether increasing C alone can remove the need for resolving-work scheduling. The two panels separate service recovery from attack work to avoid dual-axis ambiguity. Moving from $C = 8$ to $C = 64$ materially improves legitimate resolution, but attack amplification remains in the same 40 k–45 k AES-equiv./min band because unknown misses still consume bounded host work. Only the theoretical upper bound ($C = N = 512$) removes the problem entirely by eliminating fallback for all bonded identities.

The dashed P3-Persist reference shows why scheduling remains necessary for commodity low-capacity controllers even when moderate capacity expansion is available.

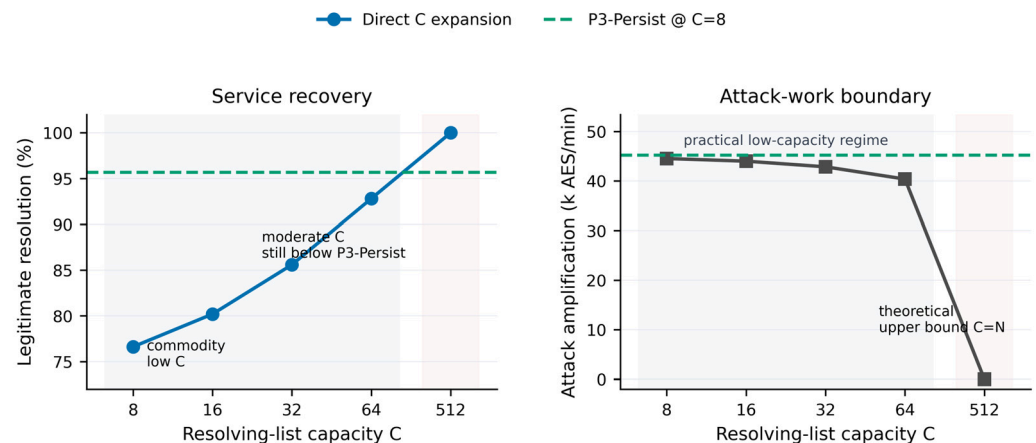


Figure 13. Direct capacity-expansion boundary: service improves as C grows, but only the theoretical upper bound ($C = N$) removes host fallback completely; low-capacity controllers still need scheduling.

6.3. RQ3: Ablation and Cache Separation

Figure 14 reports two complementary ablation views. The left panel uses Method/ablation variant on the x-axis and shows that disabling the DoS budget causes the largest work-amplification regression under heavy flood. The right panel adds the P3-Persist comparison, showing that the persistence reserve is the service-recovery layer once the work budget has bounded host scans.

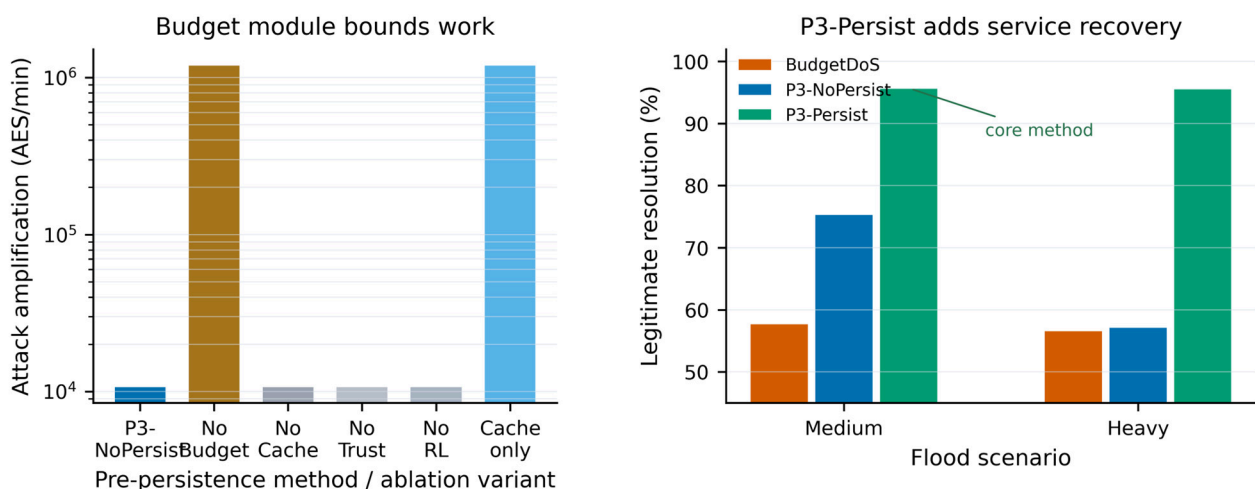


Figure 14. Method and ablation evidence: the left panel reports heavy-flood AES-equivalent work by method, while the right panel reports legitimate service recovery after adding persistence reserve.

The ablation now has two layers. The earlier module-disabled runs show that the general unresolved-work budget is necessary for amplification suppression: without it, unknown misses again drive large host-scan work. The no-reserve rows show that budgeting alone is not sufficient for strong legitimate service under sustained flood. In the 20-seed persistence comparison, P3-NoPersist remains at 0.7536 medium-flood and 0.5718 heavy-flood legitimate resolution, while P3-Persist reaches 0.9569 and 0.9557 with essentially the same attack-amplification bound. Thus, the budget is the suppression mechanism, and persistence-aware admission is the service-recovery mechanism.

6.4. Sensitivity and Correctness Boundaries

Sensitivity sweeps vary background RPA density, activity skew, RPA rotation, RSSI noise, persistence reserve R , persistence threshold k , the duplicate-filter boundary for repeated-address attack traffic, and the adaptive attacker's (m, r, p) address strategy. Figure 15 now focuses on P3-Persist, using the statistical-extension rows to show how reserve size, threshold choice, duplicate filtering, and repeated-address rate pressure affect legitimate service while attack work remains bounded in Table 18. Together, these results make the operating trade-off explicit: the general budget keeps AES-equivalent work bounded, while the persistence reserve recovers repeated legitimate RPAs after budget exhaustion. The repeated-address attacker rows should be read as boundary conditions rather than absolute victories: simple repeats are duplicate-filtered down to 504.00 AES-equivalent/min, but repeated no-filter traffic or a cross-window pool can pressure the reserve while staying below the explicit cap. The final results remain a DoS-resilience versus legitimate-deferral trade-off, not a zero-cost correctness claim. Table 20 records the correctness and delay-model boundary.

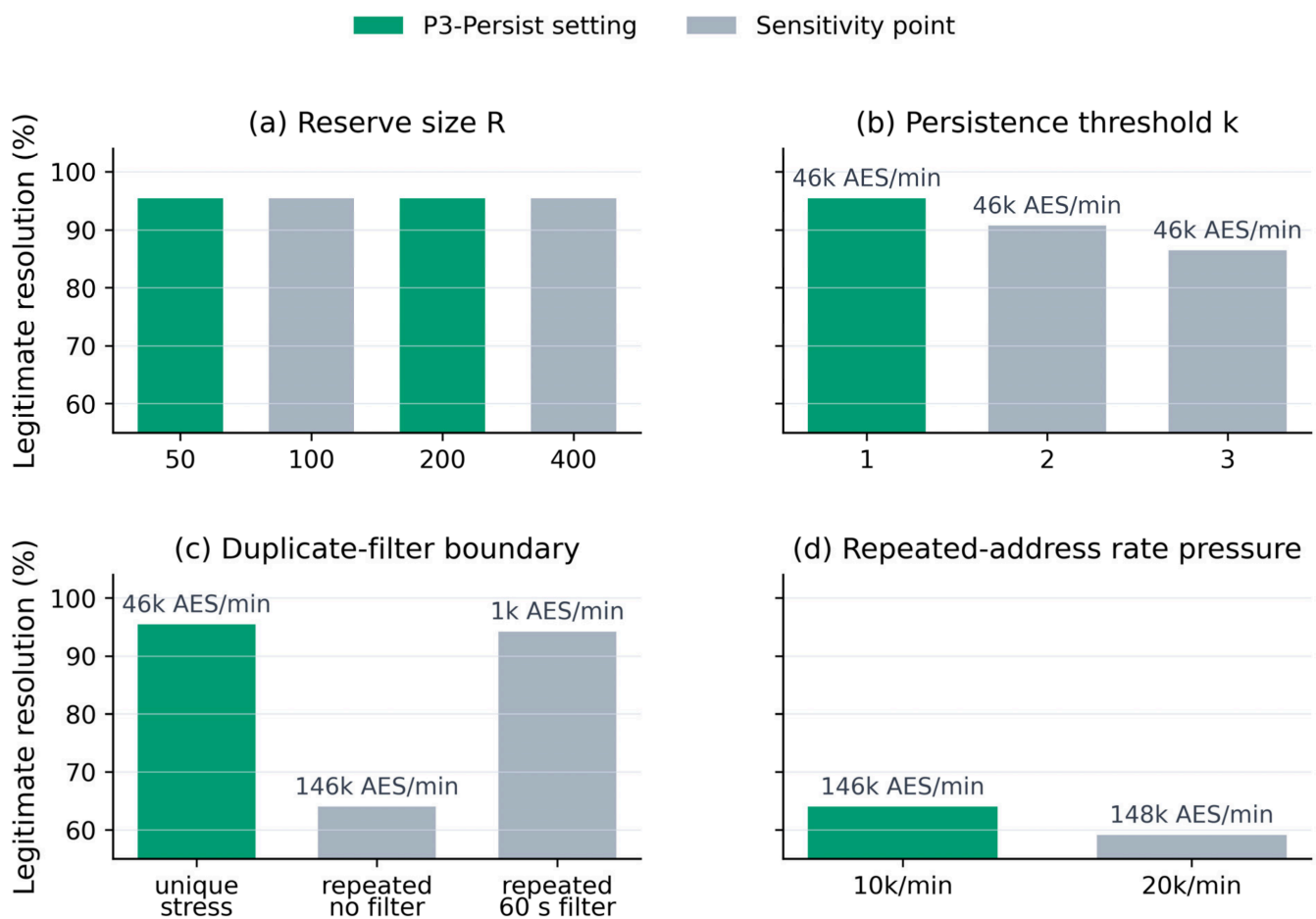


Figure 15. P3-Persist sensitivity over four factors, each panel reporting legitimate resolution (%) with the bounded attack work annotated: (a) persistence reserve size R , (b) persistence threshold k , (c) duplicate-filter boundary (unique stress, repeated no-filter, repeated 60 s filter), and (d) repeated-address rate pressure (10 k vs. 20 k attack RPAs/min). Legitimate resolution is tracked while attack work remains bounded in Table 18. Source artifacts: ‘run_m10_statistical_extension.py’, ‘run_m11_adaptive_adversary.py’, statistical-extension summaries, adaptive-sweep summaries, and figure-source CSVs listed in Table 12.

Table 20. Correctness and delay-model boundary.

Claim	Value	Boundary
Legitimate resolution rate	P3-Persist persistence-suite flood rows: 0.9569 medium, 0.9557 heavy	Simulation result, not protocol proof
False defer legitimate rate	P3-Persist persistence-suite flood rows: 0.0484 medium, 0.0497 heavy	Budget/reserve trade-off, not a zero-cost claim
Delay/latency	Not claimed from current simulator	Requires queueing or firmware/trace measurement

Table 21 reports the legitimate-resolution path mix used to interpret the scalar sensitivity metrics.

Table 21. Legitimate-resolution path mix in representative runs.

Representative Run	Method	RL Hit Rate	Cache Hit Rate	Host Scan Rate	Budget Skip Rate	Defer Rate	Interpretation
$N = 128$, $RL = 8$, no flood	StaticRL	0.740	0.000	0.260	0.000	0.000	No cache reuse; many legitimate events still pay host fallback
$N = 128$, $RL = 8$, no flood	ZephyrCache	0.010	0.950	0.040	0.000	0.000	Practical cache reuse eliminates most repeated host scans
$N = 128$, $RL = 8$, no flood	P3-NoPersist	0.000	0.950	0.050	0.000	0.000	P3 preserves the same benign fast path as ZephyrCache
$N = 1024$, $RL = 8$, 10,000 attack RPAs/min	StaticRL	0.540	0.000	0.460	0.000	0.000	Flood keeps legitimate traffic on the most expensive path
$N = 1024$, $RL = 8$, 10,000 attack RPAs/min	BudgetDoS	0.384	0.000	0.007	0.609	0.000	Budget stops host work, but denied events are uniformly dropped
$N = 1024$, $RL = 8$, 10,000 attack RPAs/min	P3-NoPersist	0.000	0.555	0.007	0.026	0.412	Same bounded host work, but denied legitimate events shift from pure drop to defer

Figure 16 visualizes the same path-mix behavior across benign reuse and heavy-flood pressure.

Table 21 and Figure 16 add a systems view that the earlier scalar metrics do not show. In the benign $N = 128$, $RL = 8$ run, the no-reserve policy looks almost identical to ZephyrCache because repeated legitimate RPAs move quickly onto the positive-cache path; this confirms that the manuscript is not hiding a benign-traffic penalty behind attack-only averages. In the heavy-flood $N = 1024$, $RL = 8$ run, P3-NoPersist and BudgetDoS keep the same near-zero legitimate host-scan fraction, but P3 redistributes denied work from pure budget_skip toward defer. P3-Persist adds the missing post-budget service-recovery path by letting repeated-address values earn reserve host scans, which is why Table 17 improves much more sharply than this pre-persistence path mix alone.

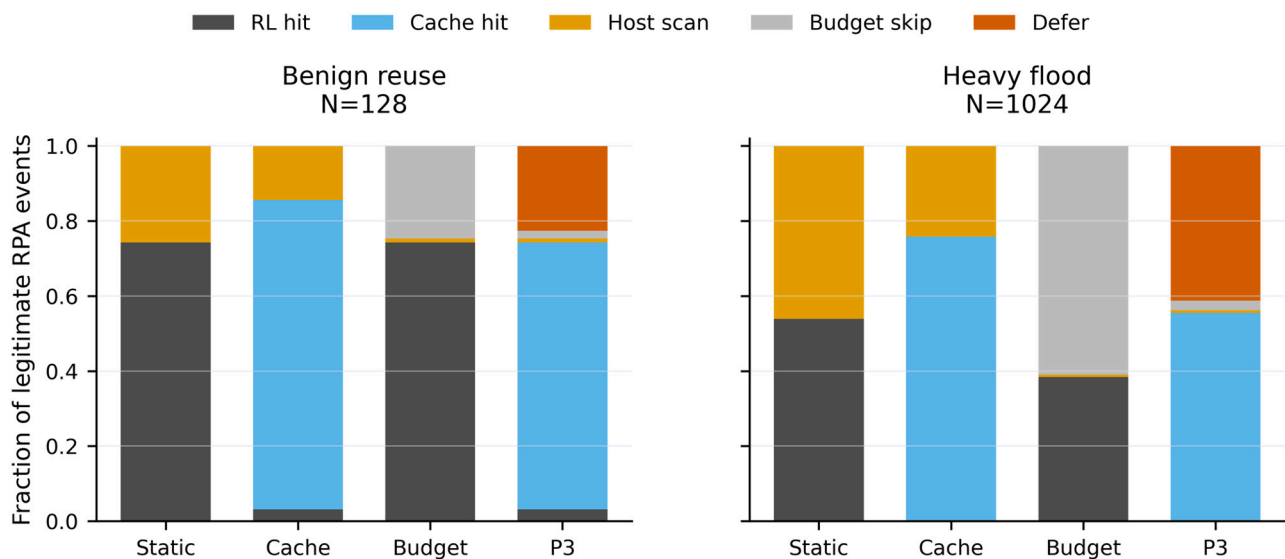


Figure 16. Legitimate path mix: stacked path fractions show whether legitimate events resolve through RL, cache, host scan, budget skip, or defer under benign reuse and heavy flood.

Figure 17 returns to the headline persistence comparisons and reports seed-level legitimate-resolution rates with Student-t 95% confidence intervals. The figure makes the main service-recovery claim visible without changing the scalar statistics in Table 17: under both medium and heavy flood, P3-Persist is consistently separated from BudgetDoS, AdaptiveRateLimit, and the pre-persistence P3-NoPersist variant, while remaining in the same bounded-amplification regime reported above.

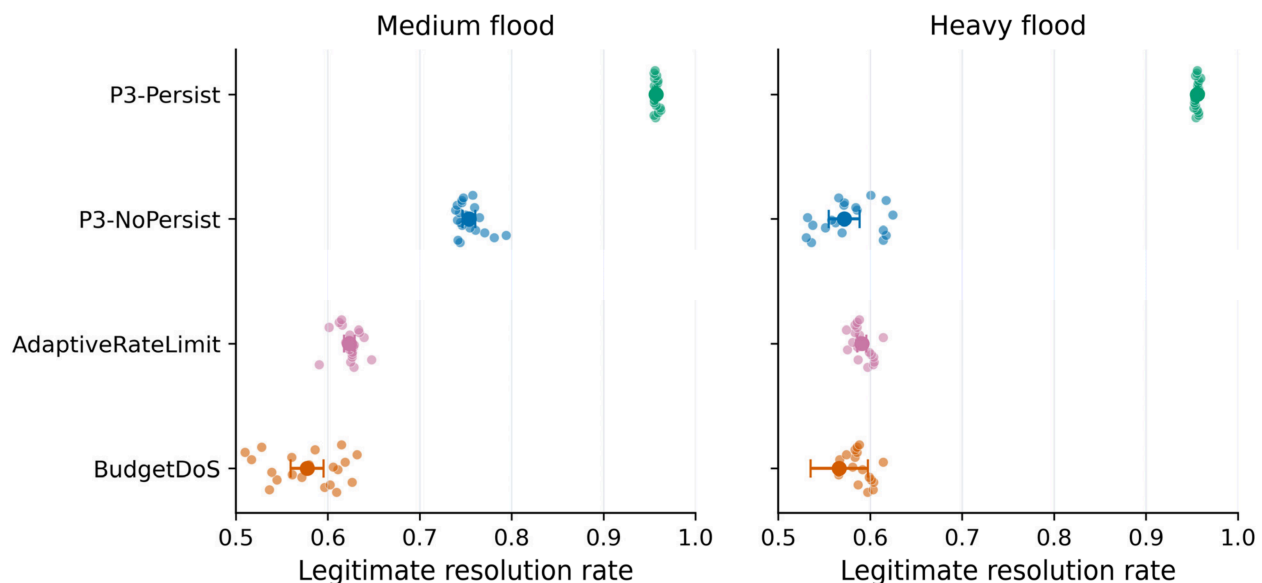


Figure 17. Seed-level legitimate-resolution rates: each point is a simulation seed and the intervals are Student-t 95% CIs for the headline persistence comparisons. Source artifacts: 'plot_legit_ci.py', 'Figure 13_legit_rate_ci' outputs, and Table 17 statistical-extension summary CSVs.

These results should be read with the stated boundary. The simulator is useful for comparing scheduling policies under controlled workload parameters. It is not a substitute for a formal proof or broad deployment study. A future hardware implementation should test bursty legitimate devices, dense public spaces, multi-radio interference, and long-term key churn. The most important practical interpretation is that P3 behaves like a load-shedding mechanism for a specific internal work queue. The method does not discard established

connections or change application-layer authorization. It limits repeated resolving attempts for traffic that has not produced useful matches. In deployment, logs should distinguish budget exhaustion from normal privacy behavior so that operators can identify sustained RPA-flooding pressure without exposing per-address decisions.

6.5. Overhead and Energy Reporting

Table 22 summarizes overhead reporting. P3-Persist adds state for cache, counters, timestamps, budget windows, and persistence counters/reserve state. The manuscript reports SRAM at the model level and energy only as AES-equivalent operation estimates. This conservative reporting prevents the most common overclaim in embedded papers: treating operation-count reduction as measured battery-life improvement.

Table 22. Overhead summary.

Item	P3-Persist Model Estimate	Interpretation
Positive cache	1024 B	64 entries $\times$ 16 B
RL metadata	32 B at RL = 8	4 B per resolving-list entry
Activity/last-seen state	4096 B at $N = 512$	8 B per bonded identity in long-window replication
Budget state	64 B	Window id, used count and counters
Persistence reserve state	1600 B at $R = 200$	Denial/reserve accounting for repeated-address values
RSSI defer hint	16 B	Internal scheduling hint only
Energy	AES-equivalent estimate only	No measured current claim
Measured power	Not claimed	Requires PPK2 or equivalent instrumentation

The 1600 B estimate for $R = 200$ assumes 8 bytes per tracked reserve entry in the model-level accounting. It is an accounting estimate for comparison, not a measured firmware allocation. The AES-equivalent unit is a modeling proxy, not a measurement of energy, CPU time, or latency. One unit denotes one modeled IRK trial computation and deliberately abstracts away platform-specific factors that a real device would incur: differences in hardware AES acceleration, host-controller transfer and interrupt costs, memory-access latency, power-management behavior, and firmware scheduling overhead. AES-equivalent attempts therefore support the fair comparison of admission policies under a common simulation model, but they must not be read as measured current, battery-life, CPU cycles, or wall-clock latency on any specific BLE platform. Translating these operation counts into on-device energy or timing is left to future firmware-level measurement (Section 7.5).

6.6. Stack-Context Auxiliary Record

Table 23 records the nRF5340 DK scanner-context and visible-address-repetition sanity check. It is not a P3 runtime, timing, energy, latency, throughput, interoperability, or production-firmware evaluation; headline policy comparisons remain simulation-based.

Table 23. nRF5340 DK auxiliary evidence.

Item	Value	Interpretation
DK enumeration	Public text omits device SNR	Board is reachable for auxiliary validation
Role	BLE observer/scanner context	Supports RSSI/noisy-advertising parameter interpretation
Boundary	Not a P3 hardware implementation	Not a real attack-defense or power measurement

7. Discussion and Threats to Validity

Table 24 reports the paired modeled-observable privacy audit, Table 25 reports the joint epoch–loss–density boundary, and Table 26 summarizes the main threats and mitigations. The evidence boundaries stated earlier (simulation-based AES-equivalent modeling, epoch-level repetition, the auxiliary nRF5340 DK record, and the modeled legitimate-deferral trade-off) keep these claims aligned with the presented evidence.

Table 24. Paired modeled-observable privacy audit (20 paired seeds per load/filter condition; external proxy = modeled defer rate, mean delay, and p95 delay).

Load	Δ (s)	P3-Persist AUC	P3-NoPersist AUC	BudgetDoS AUC	Initial Paired Audit
No attack	0	1.0	1.0	0.5	All initial decisions matched
No attack	60	0.5	0.5	0.5	All initial decisions matched
Medium flood	0	1.0	1.0	0.5	All initial decisions matched
Medium flood	60	0.5	0.5	0.5	All initial decisions matched
Heavy flood	0	0.5	0.5	0.5	All initial decisions matched
Heavy flood	60	0.5	0.5	0.5	All initial decisions matched

Table 25. P3-Persist joint epoch–loss–benign boundary (20 seeds; offered rate uses generated legitimate events, observed rate uses observed legitimate events).

Condition	Offered Legit Rate	Observed Legit Rate	Observed False Defer	Attack AES/min	Interpretation
epoch 15 min, loss 0, density 100, unique benign, attacked	0.955313	0.955313	0.050653	49,783.104	Lowest-pressure attacked main-cell reference
epoch 5 min, loss 0, density 100, unique benign, attacked	0.853841	0.853841	0.153106	49,708.512	Shorter epoch reduces service without changing the work conclusion
epoch 1 min, loss 0.4, density 5000, unique benign, attacked	0.402741	0.671334	0.333547	33,477.696	Adverse unique-benign boundary; dual denominators diverge
epoch 1 min, loss 0.4, density 5000, repeated benign, attacked	0.345137	0.575279	0.425083	33,466.608	Repeated benign values consume reserve and further reduce service
epoch 1 min, loss 0.4, density 5000, unique benign, no attack	0.406062	0.674463	0.330103	0	Epoch/loss/background pressure dominates this boundary

Table 26. Threats and mitigations.

Threat	Mitigation in Manuscript
Simulation abstraction	Claims limited to AES-equivalent work in modeled settings.
RPA repetition model	Legitimate persistence follows same-epoch RPA semantics; unique/repeated background and dense adaptive attack variants are tested in the joint robustness and adaptive-refinement experiments, but real-stack traces remain future work.
Short RPA rotation and loss	The joint robustness experiment tests 1/5/15 min epochs and 0/20/40% modeled observation loss; the report offered and observed service separately, limiting service recovery to tested conditions.
Benign unknown density	The joint robustness experiment tests 100/1000/5000 RPA/min unique-benign traffic and a repeated-benign adverse control; repeated benign reserve pressure is disclosed rather than treated as a tuning-only concern.
Deployment premise	Scope narrowed to $N > C$ retained-bond centrals; beacon-only, single-user, and $N \leq C$ deployments are excluded.
Energy estimate	Report AES-equivalent estimate, not measured current.
Hardware evidence	Keep nRF5340 DK as scanner-context record only; not validation.
Correctness	Report model-bounded observation, not formal proof.
Privacy boundary	The paired modeled-observable privacy audit supports matched initial pre-resolution decisions for identical visible traces but finds later modeled distinguishability in some conditions; no real end-to-end side-channel-free claim is made.

7.1. Privacy-Side-Channel Analysis

The most important privacy question is whether P3 creates an externally observable membership predicate. The intended external observation surfaces are the advertising channel, optional scan responses, connection attempts, timing visible to peers, application-visible behavior and logs. Proposition 0 formalizes the constraint: any pre-resolution decision whose externally visible behavior depends on whether a candidate RPA is bonded would become a membership oracle. The P3 design does not intentionally specify a bonded-identity-dependent per-address response on these surfaces before normal resolution. It does not answer whether a particular RPA is in the bonded set before the normal IRK/AES-equivalent resolving path. Persistence-aware admission observes only that the same over-the-air address value reappeared; it does not infer which identity, if any, owns that value. Budget exhaustion and reserve use are aggregate receiver-local scheduling decisions, not statements about membership [42]. The paired modeled-observable privacy audit tested this implementation property with 120 paired jobs whose timestamp, RSSI, payload length, RPA value, repetition pattern, load, and duplicate-filter setting were identical while only the offline identity label and normal resolution outcome differed. All 120 pairs had identical visible-trace hashes, and every method produced identical initial pre-resolution

decisions in every pair. This supports the identity-label invariance of the tested initial decision, not an end-to-end side-channel-free claim.

An adversary could still infer that a receiver is under load if application behavior, connection timing or logs expose budget exhaustion. This is a deployment risk rather than a cryptographic feature of P3 and is handled in the deployment guidance below. Later modeled behavior can become distinguishable after normal resolution changes cache, resolving-list, and service state. With duplicate filtering disabled, the fixed modeled delay/defer proxy produced AUC 1.0 for P3-Persist and P3-NoPersist under no/medium load, whereas heavy flood and every $\Delta = 60$ s condition produced AUC 0.5. Real scan responses, connection behavior, controller/host timing, application logs, and other end-to-end surfaces were not measured and must be evaluated in a production host stack. The AUC values in Table 24 are based on simulator-derived proxy features, not measured BLE observables; within the fixed deterministic proxy feature set, AUC = 0.5 indicates no discrimination and AUC = 1.0 indicates complete discrimination.

7.2. Short RPA Rotation and Benign Unknown Density

The joint epoch–loss–benign-density experiment directly evaluates legitimate RPA epoch, modeled observation loss, and benign unknown density without changing the $B + R$ work-bound definition. Short epochs reduced modeled service even with $k = 1$: at zero loss and density 100 RPA/min, reducing the epoch from 15 min to 5 min lowered P3-Persist offered service from 0.955313 to 0.853841. Across unique-benign cells, density had a small mean effect relative to epoch and loss, but the repeated-benign adverse control reduced offered service by 0.057604 and observed service by 0.096055 relative to the matched unique-benign cell. Deployment tuning should, therefore, monitor general-budget exhaustion, legitimate reserve use, background reserve use, and both offered- and observed-denominator service rather than scaling B and R from a single count alone. In the 20-seed main matrix, 24 of 27 cells crossed the preregistered 0.05 practical degradation screen; this threshold triggered seed extension and is not a formal non-inferiority margin. The supported conclusion is conditional service recovery: the modeled work bound can hold while legitimate service degrades under short epochs, loss, or repeated-benign reserve pressure.

7.3. Implementation Requirements and Deployment Considerations

A real implementation in a Zephyr, BlueZ, nRF-based, or other vendor-host environment would need access to four classes of state: whether an advertisement is an RPA candidate, whether the controller resolving list or positive cache already resolved it, the visible pre-resolution RPA value used for repetition counting, and the current ' B ', ' R ', ' W ', and ' k ' budget-window state. The scheduler must be placed where cache/RL misses and admission decisions are visible; if a controller or OS exposes only post-filtered events, P3-Persist cannot claim the same recovery benefit without adjusting ' k ' or moving the logic before the duplicate-filter layer. It must also keep defer behavior, scan responses, connection attempts, logs, telemetry, and application-visible delay independent of bonded membership before normal IRK resolution; otherwise, the scheduler could become a membership oracle.

7.4. Parameter Selection Guidance

The following values are simulation-informed starting hypotheses, not production defaults: B should be derived from affordable host work and tested for benign unknown load, R should balance active legitimate repeats against repeated-unknown reserve pressure, and $k = 1$ should be treated as a fast-recovery setting rather than a universal choice. The budget/persistence window W should not exceed the shortest expected legitimate RPA epoch or the known duplicate-filter window. The joint robustness experiment nevertheless shows that $W \leq \text{epoch}$ and $k = 1$ do not guarantee stable service under short

epochs and loss. Operators should monitor general-budget exhaustion, attack reserve use, background reserve use, and both offered- and observed-denominator service: an adaptive or repeated-benign pool can remain within the $B + R$ cap while materially degrading service or consuming reserve. In the present simulation matrix, Table 18 shows that ' $R = 50$ ' already saturates the ' $N = 512$ ' repeated-legitimate workload, Table 18 brackets the ' $k = 1$ ' versus ' $k = 2$ ' trade-off, and Table 16 provides the budget frontier used to scale ' B '. These values do not establish epoch = 15 min or any tested $B/R/W/k$ tuple as a production default or global optimum; deployment-specific choices require trace-based or firmware-level validation.

These parameter choices do not change BLE advertising formats, pairing procedures, key derivation, or peer behavior. The trade-off is that P3 cannot prevent an attacker from occupying airtime or generating radio noise. It only reduces the receiver-side resolving effort induced by unknown private-address candidates. This narrower claim is still useful because many embedded systems fail under load before the radio channel is fully saturated.

7.5. Future Work

Future work should implement P3-Persist in an open BLE host stack and test it with physical advertising traffic, multiple bonded peripherals, controller duplicate filtering, calibrated current measurement, deferral duration, queueing delay, and time-to-resolution. A second direction is to evaluate the adaptive tuning of B , R , and k under real public-space density. A third direction is to compare the method across representative open and vendor BLE stacks where resolving-list capacity and host/controller split differ. These extensions would strengthen external validity but are beyond the current simulation-supported manuscript package.

8. Conclusions

BLE private-address resolution is both a privacy mechanism and a resource-management path. When resolving-list capacity is limited and unknown-RPA traffic is high, resolving work can become a denial-of-service surface. This paper contributes P3-Persist, which is a persistence-aware resolving admission policy for that surface, evaluated as a simulation-based and stack-informed scheduling design rather than as a deployed firmware implementation. P3 first bounds cache/RL-miss host work with an unresolved-work budget, then uses a small reserve for repeated over-the-air RPA values after budget exhaustion. The mechanism uses visible address repetition rather than an intentional bonded-identity label for initial pre-resolution admission. The paired simulation supports this initial-decision invariance but does not establish that later or real end-to-end behavior is side-channel free. In 20-seed, 1800 s simulations, P3-Persist keeps unique-flood attack amplification in the same approximately 45 k AES-equivalent/min bounded regime as BudgetDoS and AdaptiveRateLimit, while raising legitimate resolution above the generic limiters reported in Section 6. Within the configured simulation matrix, the reported legitimate-service comparisons have non-overlapping 95% CIs and paired $p < 0.01$.

The dense adaptive refinement replaces the earlier coarse-grid single-point result with a tied local region near $m = 320$ and $r = 1$. The highest confirmed mean is 146,196.29 AES-equivalent/min (96.69% of the 151,200 cap), and the maximum seed is 147,006.72; the modeled cap remains unexceeded, but the remaining headroom is small. Service recovery is conditional rather than universal: 24/27 joint epoch-loss-density cells cross the practical degradation screen and repeated benign values can consume reserve even while the work bound holds. The contribution is a reproducible, privacy-constrained scheduling design and simulation evaluation framework for BLE RPA-resolving work. Firmware integration, real end-to-end privacy and IRK-ground-truth evaluation, broader adaptive-attacker search,

queueing behavior, and on-device cost measurement remain necessary before deployment effectiveness can be claimed.

Supplementary Materials: The following supporting information can be downloaded at <https://www.mdpi.com/article/10.3390/electronics15173889/s1>. Table S1: Complete long-window validation summary (18-row method-by-scenario table for N = 512 and RL = 8).

Author Contributions: Conceptualization, S.C., L.W. and T.-T.N.; methodology, S.C. and T.-T.N.; software, S.C.; validation, S.C., T.-T.N. and T.-K.D.; formal analysis, S.C.; investigation, S.C. and L.W.; resources, T.-T.N. and L.W.; data curation, S.C.; writing—original draft preparation, S.C.; writing—review and editing, S.C., T.-T.N. and T.-K.D.; visualization, S.C.; supervision, T.-T.N. and T.-K.D.; project administration, T.-T.N. and S.C.; funding acquisition, S.C. and T.-T.N. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The headline policy-comparison datasets in this study are generated by an open, deterministic simulator. In addition, the supplementary artifact includes an anonymized nRF5340 DK passive advertising trace used only to sanity-check visible private-address repetition; the hardware trace is not used for headline performance claims. Controller duplicate filtering was disabled during hardware capture, and the filter-on comparisons are offline replays of the same captured event stream rather than controller measurements. The simulation source code (rpa_sim.py), the paired modeled-observable privacy-audit, joint-robustness, and adaptive-refinement experiment and analysis scripts, configuration files, random-seed records, per-run manifests, paired-trace audits, generated/observed/dropped metrics, observation-loss and repeated-benign capability tests, complete local-search tables, summary and contrast tables, cap/closure audits, figure-source CSV files, worked-example traces, anonymized hardware-trace events, repeatability summaries, and offline duplicate-filter replay tables are available to editors and reviewers through the supplementary artifact package or restricted repository/archive access provided by the corresponding author. The versioned artifact package is publicly available at <https://github.com/shenchongcn/p3-ble-rpa-dos/releases/tag/v1.0.0> (accessed on 10 July 2026) and is permanently archived on Zenodo at <https://doi.org/10.5281/zenodo.21521522>. The package excludes raw BLE addresses, board identifiers, exact wall-clock times, location data, private local paths, account material, screenshots, and patent-sensitive implementation detail not needed to reproduce the reported results.

Acknowledgments: The authors acknowledge the official Bluetooth, Zephyr/NCS, and BlueZ documentation ecosystems used for implementation-context analysis.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AES	Advanced Encryption Standard; used in this paper as AES-equivalent resolving work
BLE	Bluetooth Low Energy
CI	Confidence Interval
CSV	Comma-Separated Values
DK	Development Kit
DoS	Denial of Service
GATT	Generic Attribute Profile
IRK	Identity Resolving Key
LRU	Least Recently Used
NCS	nRF Connect SDK
NIST	National Institute of Standards and Technology
P3	Method label for the persistence-aware BLE RPA resolving scheduler family; P3 is not an acronym

PPK2	Nordic Power Profiler Kit II
RFC	Request for Comments
RL	Resolving List
RPA	Resolvable Private Address
RQ	Research Question
RSSI	Received Signal Strength Indicator
SD	Standard Deviation
SNR	Serial Number
SRAM	Static Random-Access Memory
TTL	Time To Live

References

1. Tosi, J.; Taffoni, F.; Santacatterina, M.; Sannino, R.; Formica, D. Performance Evaluation of Bluetooth Low Energy: A Systematic Review. *Sensors* **2017**, *17*, 2898. [CrossRef] [PubMed]
2. Al-Shareeda, M.A.; Saare, M.A.; Manickam, S.; Karuppayah, S. Bluetooth Low Energy for Internet of Things: Review, Challenges, and Open Issues. *Indones. J. Electr. Eng. Comput. Sci.* **2023**, *31*, 1182–1189. [CrossRef]
3. Gomez, C.; Oller, J.; Paradells, J. Overview and Evaluation of Bluetooth Low Energy: An Emerging Low-Power Wireless Technology. *Sensors* **2012**, *12*, 11734–11753. [CrossRef]
4. Gautam, S.; Kumar, S.; Tsiropoulou, E.E. Unlocking Ultralow-Power Bluetooth Low Energy Relays with Periodic Advertisements. *IEEE Internet Things J.* **2026**, *13*, 27717–27735. [CrossRef]
5. Bluetooth SIG. *2025 Market Update*; Bluetooth SIG: Kirkland, WA, USA, 2025. Available online: <https://www.bluetooth.com/2025-market-update> (accessed on 10 July 2026).
6. Lacava, A.; Zottola, V.; Bonaldo, A.; Cuomo, F.; Basagni, S. Securing Bluetooth Low Energy Networking: An Overview of Security Procedures and Threats. *Comput. Netw.* **2022**, *211*, 108953. [CrossRef]
7. Celosia, G.; Cunche, M. Saving Private Addresses: An Analysis of Privacy Issues in the Bluetooth-Low-Energy Advertising Mechanism. In Proceedings of the 16th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services, Houston, TX, USA, 12–14 November 2019; pp. 444–453. [CrossRef]
8. Bluetooth SIG. *Bluetooth Core Specification*; Version 6.3, Vol. 6, Part B, Section 1.3.2.2.2, Resolvable Private Address, and Vol. 4, Part E, LE Read Resolving List Size Command; Bluetooth SIG: Kirkland, WA, USA, 2026. Available online: <https://www.bluetooth.com/specifications/specs/core-specification-6-3> (accessed on 10 July 2026).
9. Padgett, J.; Bahr, J.; Batra, M.; Holtmann, M.; Smithbey, R.; Chen, L.; Scarfone, K. *Guide to Bluetooth Security*; NIST Special Publication 800-121 Revision 2 Update 1; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2022. [CrossRef]
10. Zephyr Project. Stack Architecture. Available online: <https://docs.zephyrproject.org/latest/services/connectivity/bluetooth/bluetooth-arch.html> (accessed on 10 July 2026).
11. Zephyr Project. Identity Resolving Key Lookup Source Code. Available online: <https://github.com/zephyrproject-rtos/zephyr/blob/main/subsys/bluetooth/host/keys.c> (accessed on 10 July 2026).
12. Zephyr Project. CONFIG_BT_CTLR_RL_SIZE Bluetooth Controller Resolving List Size Kconfig Option. Available online: https://docs.zephyrproject.org/latest/kconfig.html#CONFIG_BT_CTLR_RL_SIZE (accessed on 10 July 2026).
13. Zhang, Y.; Lin, Z. When Good Becomes Evil: Tracking Bluetooth Low Energy Devices via Allowlist-Based Side Channel and Its Countermeasure. In Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, Los Angeles, CA, USA, 7–11 November 2022; pp. 3181–3194. [CrossRef]
14. Zhang, Y.; Weng, J.; Dey, R.; Fu, X. Bluetooth Low Energy (BLE) Security and Privacy. In *Encyclopedia of Wireless Networks*; Springer International Publishing: Berlin/Heidelberg, Germany, 2020; pp. 123–134. [CrossRef]
15. Sun, D.; Tian, Y. Address Privacy of Bluetooth Low Energy. *Mathematics* **2022**, *10*, 4346. [CrossRef]
16. Barua, A.; Al Alamin, M.A.; Hossain, M.S.; Hossain, E. Security and Privacy Threats for Bluetooth Low Energy in IoT and Wearable Devices: A Comprehensive Survey. *IEEE Open J. Commun. Soc.* **2022**, *3*, 251–281. [CrossRef]
17. Martin, J.; Alpuche, D.; Bodeman, K.; Brown, L.; Fenske, E.; Foppe, L.; Mayberry, T.; Rye, E.C.; Sipes, B.; Teplov, S. Handoff All Your Privacy: A Review of Apple’s Bluetooth Low Energy Continuity Protocol. *Proc. Priv. Enhancing Technol.* **2019**, *2019*, 34–53. [CrossRef]
18. Wu, J.; Traynor, P.; Xu, D.; Tian, D.J.; Bianchi, A. Finding Traceability Attacks in the Bluetooth Low Energy Specification and Its Implementations. In Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24), Philadelphia, PA, USA, 14–16 August 2024; pp. 4499–4516.
19. Broder, A.; Mitzenmacher, M. Network Applications of Bloom Filters: A Survey. *Internet Math.* **2004**, *1*, 485–509. [CrossRef]

20. BlueZ Project. BlueZ: Official Linux Bluetooth Protocol Stack Documentation. Available online: <https://www.bluez.org/> (accessed on 10 July 2026).
21. Nordic Semiconductor. nRF Connect SDK v3.2.1 Documentation. Available online: https://nrfconnectdocs.nordicsemi.com/ncs/3.2.1/nrf/installation/install_ncs.html (accessed on 10 July 2026).
22. Heinanen, J.; Guerin, R. A Single Rate Three Color Marker; RFC 2697; RFC Editor, September 1999. Available online: <https://www.rfc-editor.org/info/rfc2697> (accessed on 10 July 2026).
23. Das, A.K.; Pathak, P.H.; Chuah, C.-N.; Mohapatra, P. Uncovering Privacy Leakage in BLE Network Traffic of Wearable Fitness Trackers. In Proceedings of the 17th International Workshop on Mobile Computing Systems and Applications, St. Augustine, FL, USA, 23–24 February 2016; pp. 99–104. [\[CrossRef\]](#)
24. Becker, J.K.; Li, D.; Starobinski, D. Tracking Anonymized Bluetooth Devices. *Proc. Priv. Enhancing Technol.* **2019**, 2019, 50–65. [\[CrossRef\]](#)
25. Fawaz, K.; Kim, K.-H.; Shin, K.G. Protecting Privacy of BLE Device Users. In Proceedings of the 25th USENIX Security Symposium (USENIX Security 16), Austin, TX, USA, 10–12 August 2016; pp. 1205–1221. Available online: <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/fawaz> (accessed on 10 July 2026).
26. Issoufaly, T.; Tournoux, P.U. BLEB: Bluetooth Low Energy Botnet for Large Scale Individual Tracking. In Proceedings of the 2017 1st International Conference on Next Generation Computing Applications (NextComp), Mauritius, 19–21 July 2017; pp. 115–120. [\[CrossRef\]](#)
27. Garbelini, M.E.; Wang, C.; Chattopadhyay, S.; Sun, S.; Kurniawan, E. SweynTooth: Unleashing Mayhem over Bluetooth Low Energy. In Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC 20), Boston, MA, USA, 15–17 July 2020; pp. 911–925. Available online: <https://www.usenix.org/conference/atc20/presentation/garbelini> (accessed on 10 July 2026).
28. Wu, J.; Nan, Y.; Kumar, V.; Tian, D.J.; Bianchi, A.; Payer, M.; Xu, D. BLESa: Spoofing Attacks against Reconnections in Bluetooth Low Energy. In Proceedings of the 14th USENIX Workshop on Offensive Technologies (WOOT 20), Online, 11 August 2020. Available online: <https://www.usenix.org/conference/woot20/presentation/wu> (accessed on 10 July 2026).
29. Wu, J.; Nan, Y.; Kumar, V.; Payer, M.; Xu, D. BlueShield: Detecting Spoofing Attacks in Bluetooth Low Energy Networks. In Proceedings of the 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020), Fully Virtually, 14–16 October 2020; pp. 397–411. Available online: <https://www.usenix.org/conference/raid2020/presentation/wu> (accessed on 10 July 2026).
30. Ghorri, M.R.; Wan, T.-C.; Sodhy, G.C. Bluetooth Low Energy Mesh Networks: Survey of Communication and Security Protocols. *Sensors* **2020**, 20, 3590. [\[CrossRef\]](#) [\[PubMed\]](#)
31. Che, X.; He, Y.; Feng, X.; Sun, K.; Xu, K.; Li, Q. BlueSWAT: A Lightweight State-Aware Security Framework for Bluetooth Low Energy. In Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security, Salt Lake City, UT, USA, 14–18 October 2024; pp. 2087–2101. [\[CrossRef\]](#)
32. Kindt, P.H.; Yunge, D.; Diemer, R.; Chakraborty, S. Energy Modeling for the Bluetooth Low Energy Protocol. *ACM Trans. Embed. Comput. Syst.* **2020**, 19, 1–32. [\[CrossRef\]](#)
33. Nikodem, M.; Bawiec, M. Experimental Evaluation of Advertisement-Based Bluetooth Low Energy Communication. *Sensors* **2020**, 20, 107. [\[CrossRef\]](#) [\[PubMed\]](#)
34. Floyd, S.; Jacobson, V. Random Early Detection Gateways for Congestion Avoidance. *IEEE/ACM Trans. Netw.* **1993**, 1, 397–413. [\[CrossRef\]](#)
35. Schiefelbein, M.C.; St Germain, J.; Mazzola, S.L.; Gervin, R.C. Bluetooth Low Energy (BLE) Data Collection for COVID-19 Exposure Notification; MIT Lincoln Laboratory. 2022. Available online: <https://www.ll.mit.edu/r-d/publications/bluetooth-low-energy-ble-data-collection-covid-19-exposure-notification> (accessed on 10 July 2026).
36. Dao, T.K.; Nguyen, T.T. A review of the bat algorithm and its varieties for industrial applications. *J. Intell. Manuf.* **2025**, 36, 5327–5349. [\[CrossRef\]](#)
37. Assayag, Y.; Oliveira, H.; Lima, M.; Junior, J.; Preste, M.; Guimarães, L.; Souto, E. Indoor Environment Dataset Based on RSSI Collected with Bluetooth Devices. *Data Brief* **2024**, 55, 110692. [\[CrossRef\]](#) [\[PubMed\]](#)
38. Sikeridis, D.; Papapanagiotou, I.; Devetsikiotis, M. BLEBeacon: A Real-Subject Trial Dataset from Mobile Bluetooth Low Energy Beacons. *arXiv* **2018**, arXiv:1802.08782. Available online: <https://arxiv.org/abs/1802.08782> (accessed on 10 July 2026).
39. Nguyen, T.-T.; Dao, T.-K.; Nguyen, V.-T.; Pham, D.-T. Designing Effective Drug Therapies Using a Multiobjective Spider-Wasp Optimizer. *Biomimetics* **2025**, 10, 219. [\[CrossRef\]](#) [\[PubMed\]](#)
40. Strzoda, A.; Grochla, K.; Polys, K. Variability of BLE Advertisement Packets Received Signal Strength and Delivery Probability in the Presence of Interferences. In Proceedings of the 12th ACM International Symposium on Design and Analysis of Intelligent Vehicular Networks and Applications, Montreal, QC, Canada, 24–28 October 2022; pp. 37–44. [\[CrossRef\]](#)

41. Bluetooth SIG. Bluetooth Specifications and Test Documents. Available online: <https://www.bluetooth.com/specifications/specs> (accessed on 10 July 2026).
42. Nguyen, T.-T.; Ngo, T.G.; Chu, S.C.; Dao, T.K.; Nguyen, T.T.T. Recent Digital Systems Information Hiding Techniques via Internet Technology: A Review. *J. Internet Technol.* **2025**, *26*, 641–652. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.